

Generalized Evidence Passing for Effect Handlers

Efficient Compilation of Effect Handlers to C



Ningning Xie



香港大學
THE UNIVERSITY OF HONG KONG

Daan Leijen

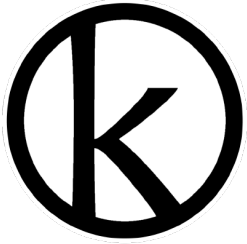


Microsoft®
Research



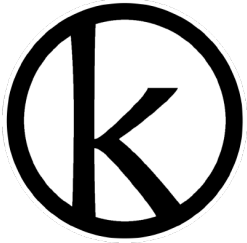
ICFP 2021

Research contributions



<https://koka-lang.github.io/>

Research contributions



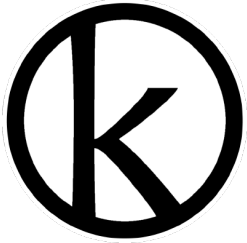
<https://koka-lang.github.io/>

Algebraic effects

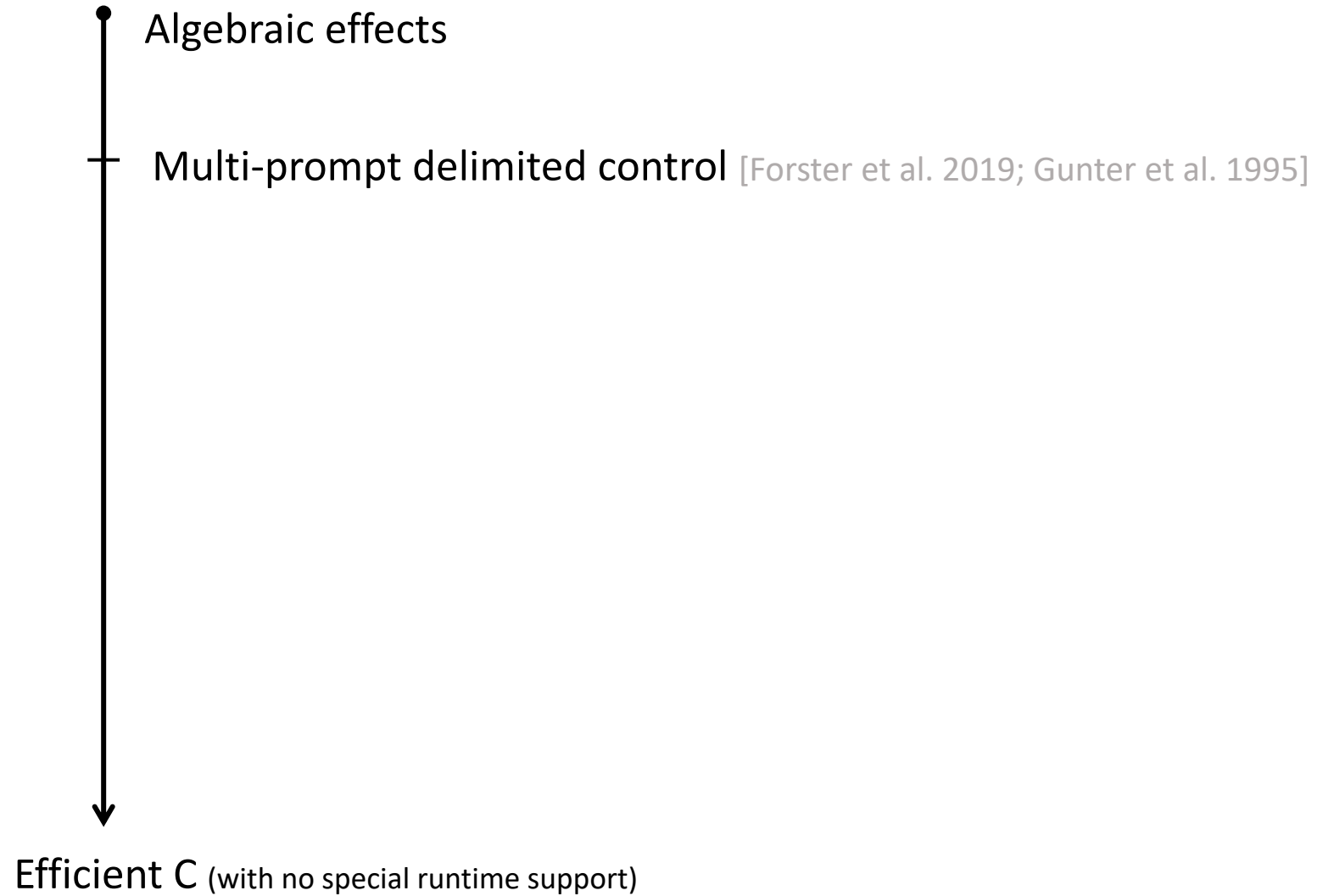


Efficient C (with no special runtime support)

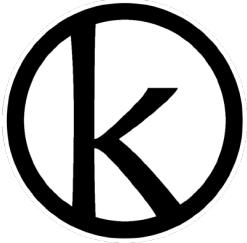
Research contributions



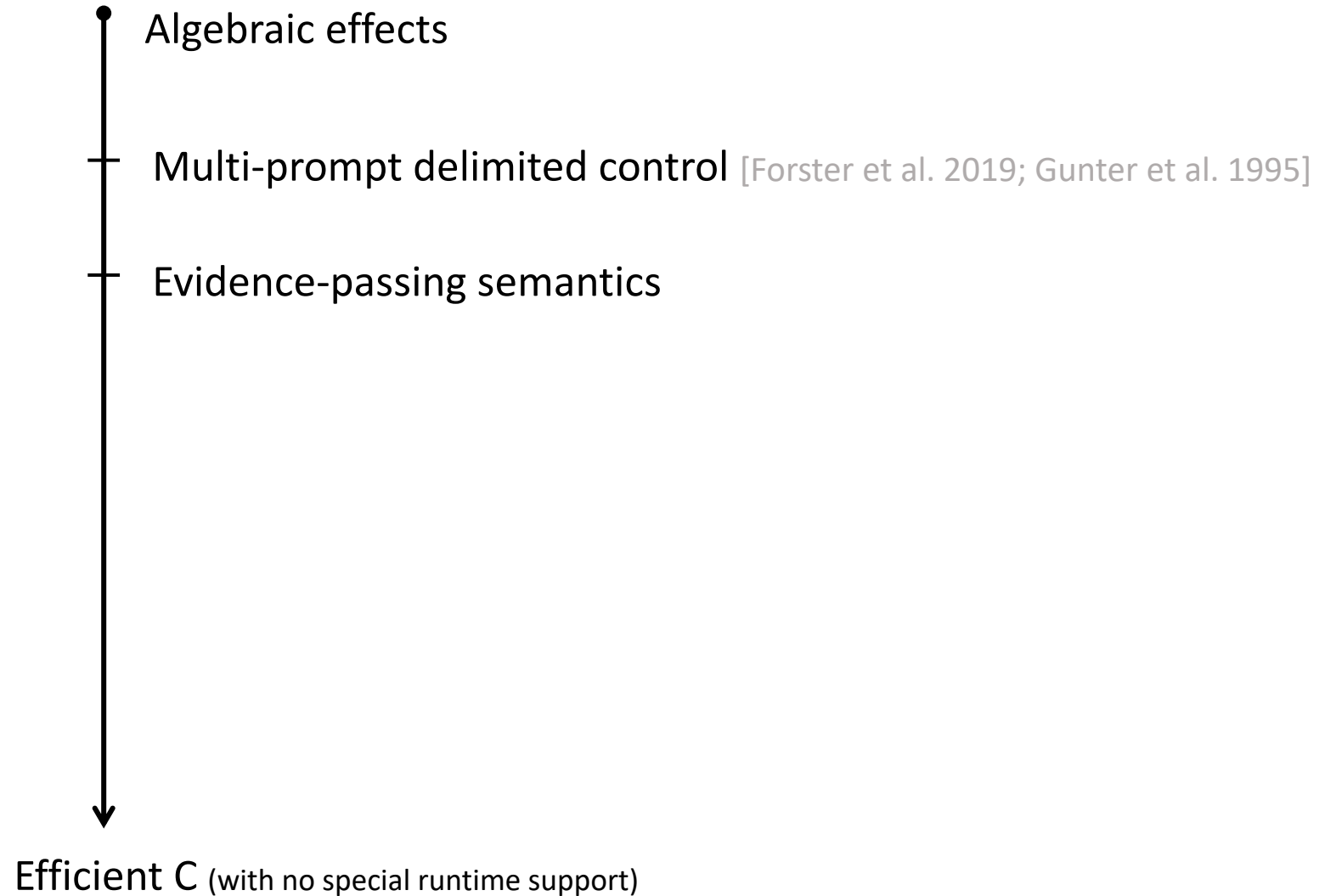
<https://koka-lang.github.io/>



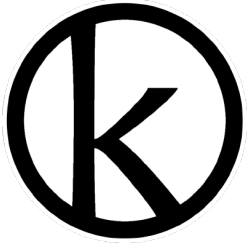
Research contributions



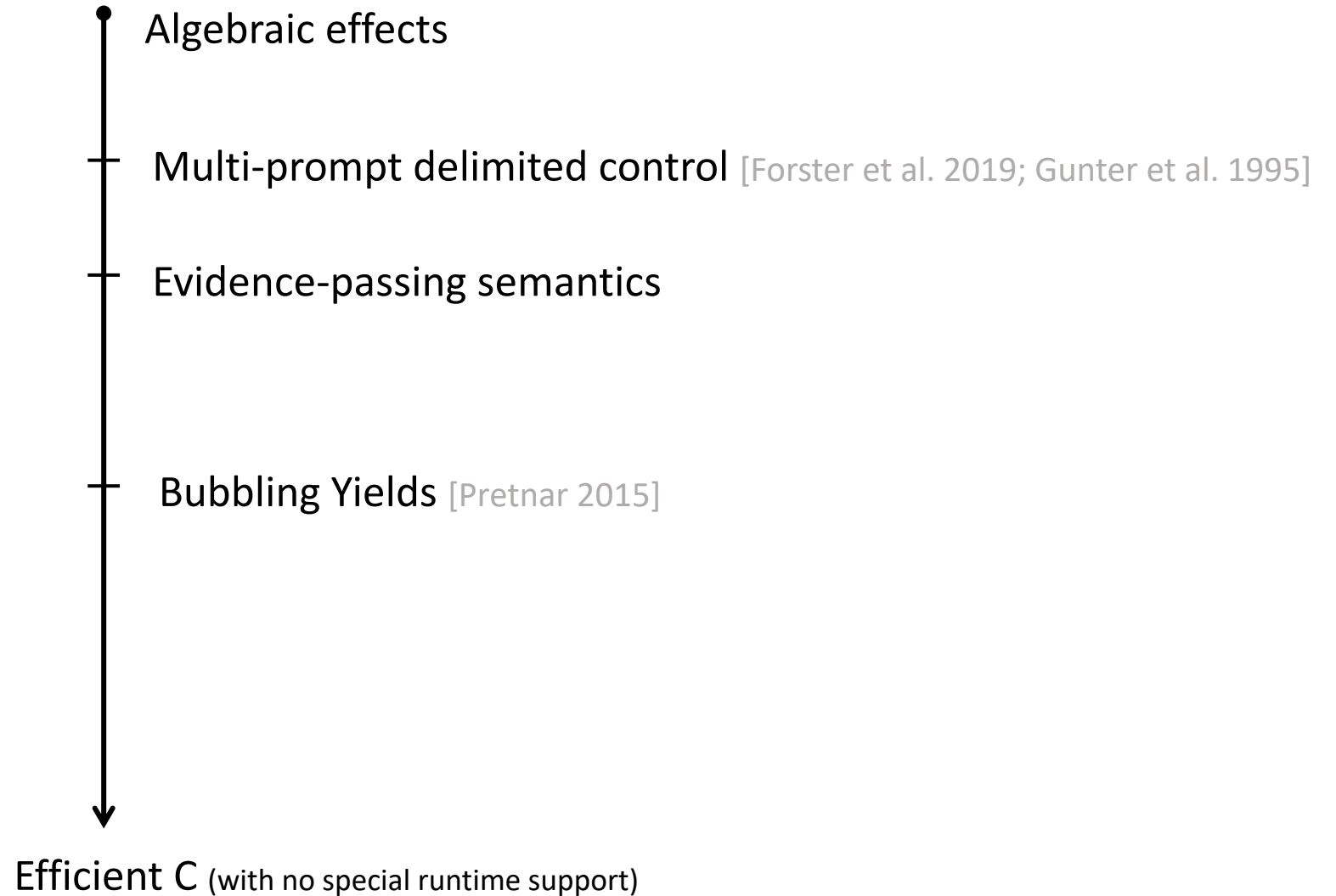
<https://koka-lang.github.io/>



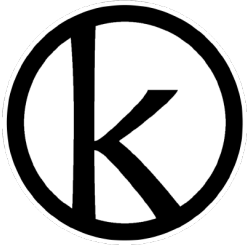
Research contributions



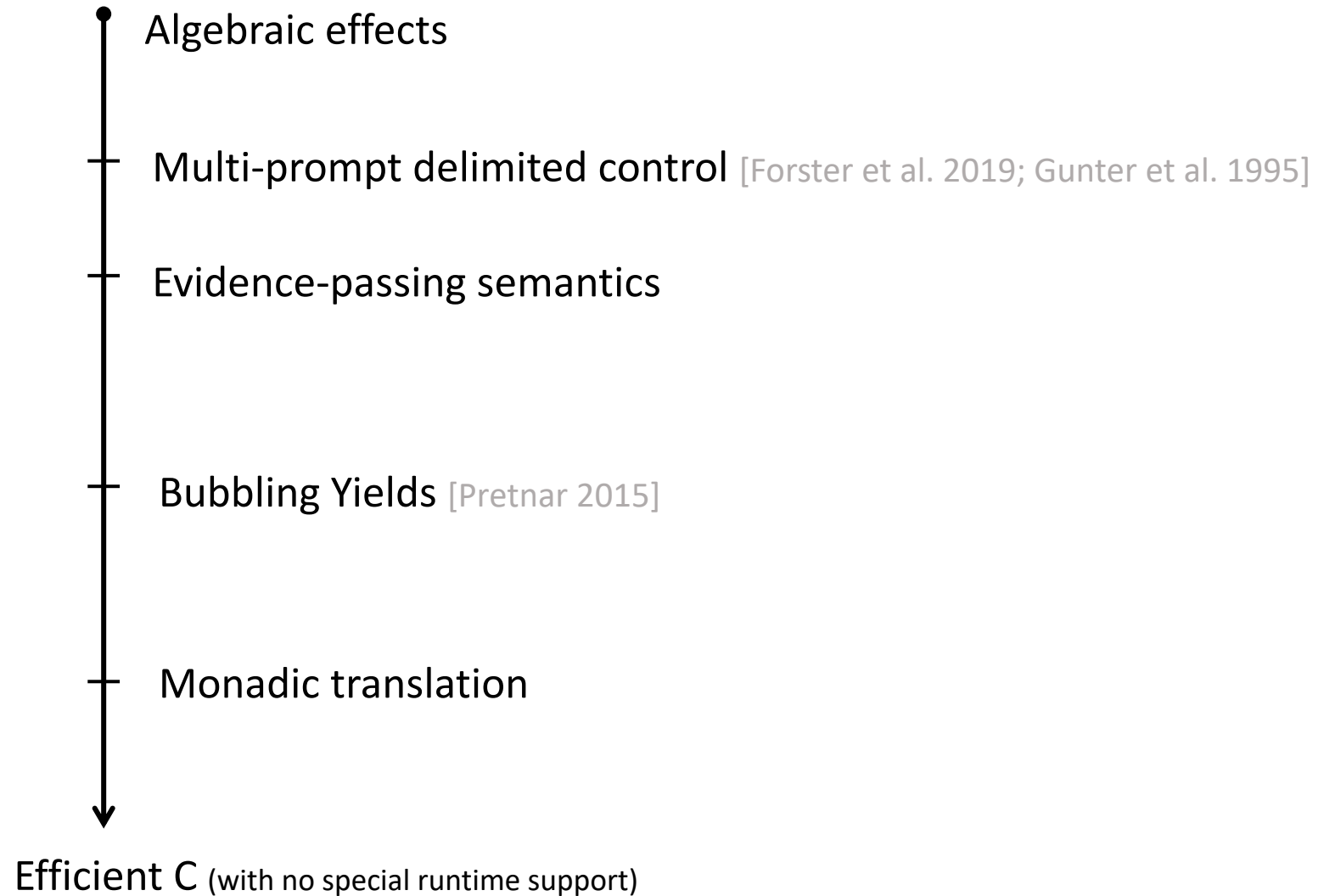
<https://koka-lang.github.io/>



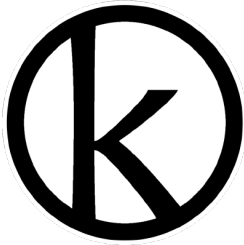
Research contributions



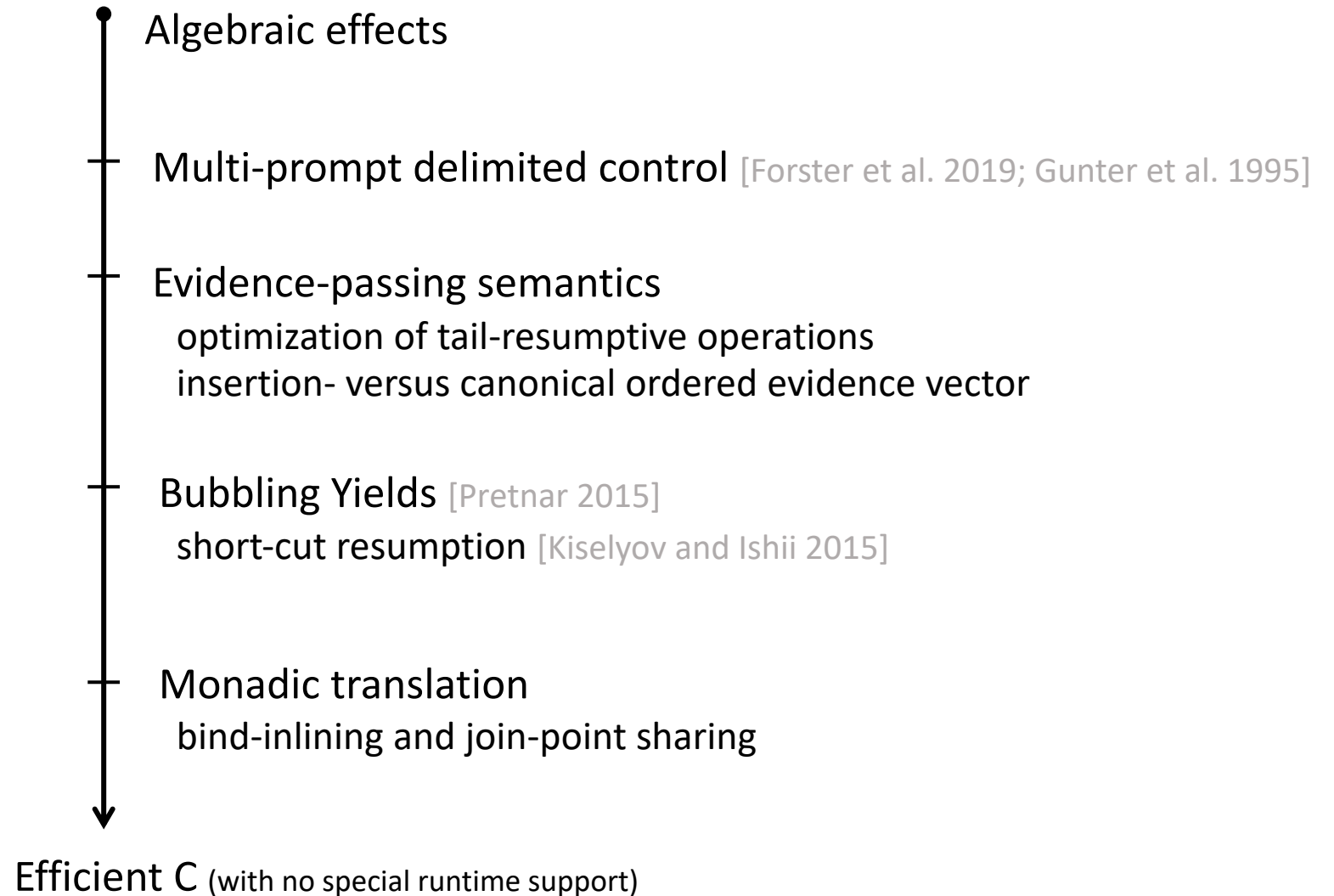
<https://koka-lang.github.io/>



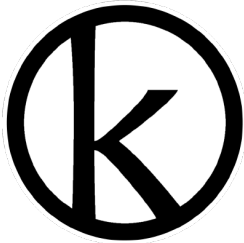
Research contributions



<https://koka-lang.github.io/>



Research contributions



<https://koka-lang.github.io/>

PLDI 2021

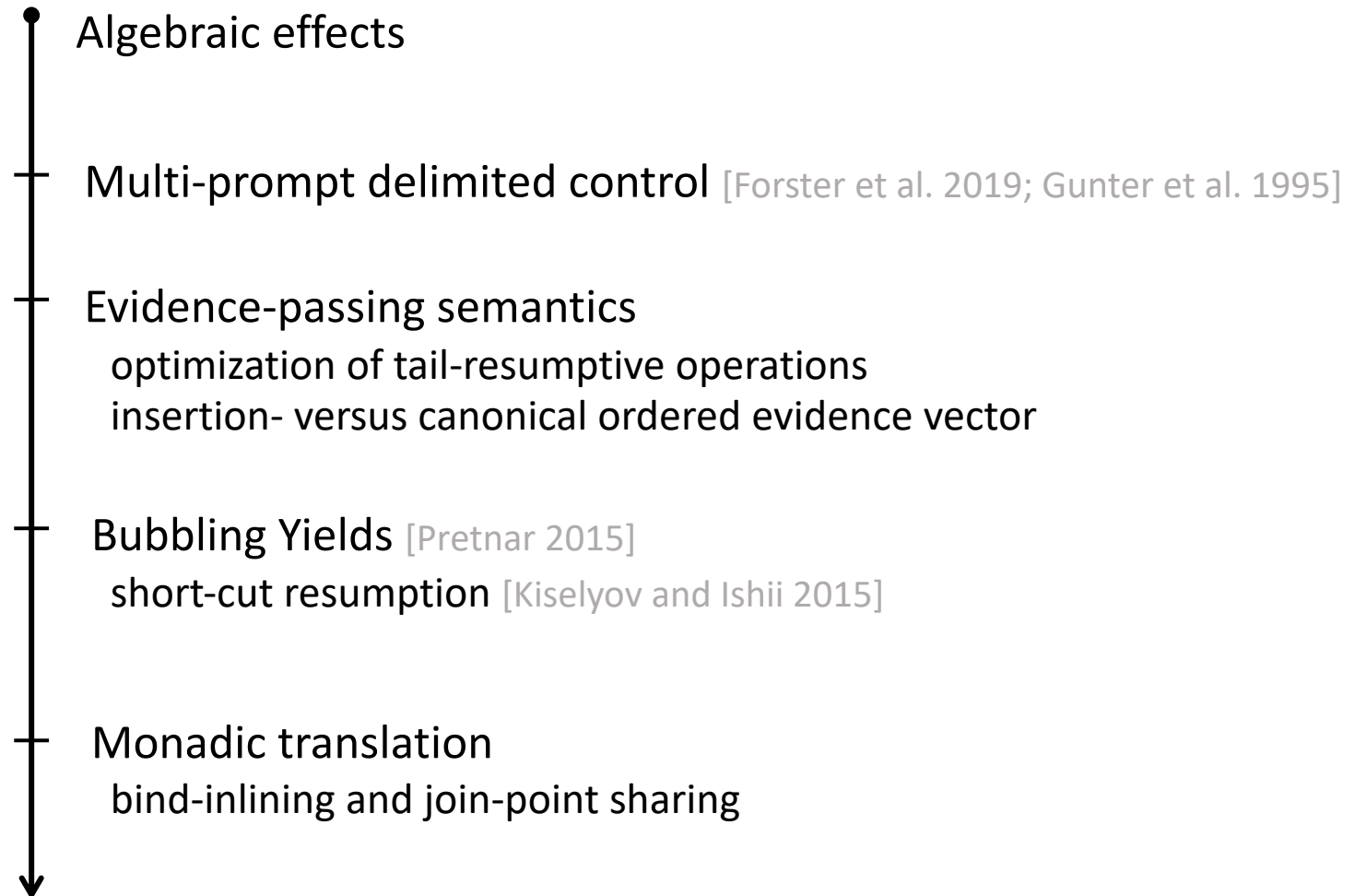
Perceus: Garbage Free Reference Counting with Reuse

Alex Reinking*
Microsoft Research
Redmond, WA, USA
alex_reinking@berkeley.edu

Leonardo de Moura
Microsoft Research
Redmond, WA, USA
leonardo@microsoft.com

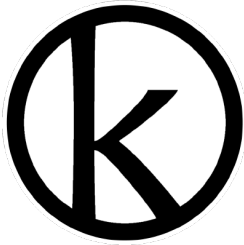
Ningning Xie*
University of Hong Kong
Hong Kong, China
nnxie@cs.hku.hk

Daan Leijen
Microsoft Research
Redmond, WA, USA
daan@microsoft.com



Efficient C (with no special runtime support)

Research contributions



<https://koka-lang.github.io/>

PLDI 2021

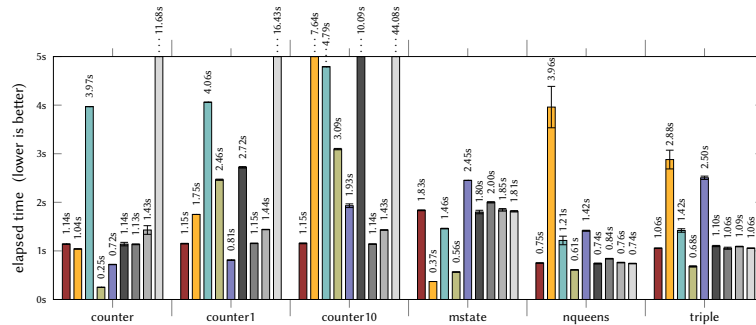
Perceus: Garbage Free Reference Counting with Reuse

Alex Reinking*
Microsoft Research
Redmond, WA, USA
alex_reinking@berkeley.edu

Leonardo de Moura
Microsoft Research
Redmond, WA, USA
leonardo@microsoft.com

Ningning Xie*
University of Hong Kong
Hong Kong, China
nnxie@cs.hku.hk

Daan Leijen
Microsoft Research
Redmond, WA, USA
daan@microsoft.com



Algebraic effects

Multi-prompt delimited control [Forster et al. 2019; Gunter et al. 1995]

Evidence-passing semantics

optimization of tail-resumptive operations

insertion- versus canonical ordered evidence vector

Bubbling Yields [Pretnar 2015]

short-cut resumption [Kiselyov and Ishii 2015]

Monadic translation

bind-inlining and join-point sharing

Efficient C (with no special runtime support)

Algebraic effects 101

```
effect read {  
  ask : () -> int  
}
```

```
handler {  
  ask -> \x.\k. k 1  
}  
(\_.  
  perform ask () + perform ask ()  
)
```

Algebraic effects 101

```
effect read {  
  ask : () -> int  
}
```

```
handler {  
  ask -> \x.\k. k 1  
}  
(\_.  
  perform ask () + perform ask ()  
)
```

Algebraic effects 101

```
effect read {  
  ask : () -> int  
}
```

```
handler {  
  ask -> \x.\k. k 1  
}
```

```
(\_.  
  perform ask () + perform ask ()  
)
```

Algebraic effects 101

```
effect read {  
  ask : () -> int  
}
```

```
handler {  
  ask -> \x.\k. k 1  
}
```

```
(\_.  
  perform ask () + perform ask ()  
)
```

Algebraic effects 101

effect

```
effect read {  
  ask : () -> int  
}
```

```
handler {  
  ask -> \x.\k. k 1  
}
```

```
(\_.  
  perform ask () + perform ask ()  
)
```

Algebraic effects 101

effect

```
effect read {  
  ask : () -> int  
}
```

operation

```
handler {  
  ask -> \x.\k. k 1  
}
```

```
(\_.  
  perform ask () + perform ask ()  
)
```

Algebraic effects 101

effect

```
effect read {  
  ask : () -> int  
}
```

operation

effect handler

```
handler {  
  ask -> \x.\k. k 1  
}
```

```
(\_.  
  perform ask () + perform ask ()  
)
```

Algebraic effects 101

effect

```
effect read {  
  ask : () -> int  
}
```

operation

effect handler

```
handler {  
  ask -> \x.\k. k 1  
}
```

implementation

```
(\_.  
  perform ask () + perform ask ()  
)
```

Algebraic effects 101

effect

```
effect read {  
  ask : () -> int  
}
```

operation

effect handler

```
handler {  
  ask -> \x.\k. k 1  
}
```

implementation

computation

```
(\_.  
  perform ask () + perform ask ()  
)
```

Algebraic effects 101

effect

```
effect read {  
  ask : () -> int  
}
```

operation

effect handler

```
handler {  
  ask -> \x.\k. k 1  
}
```

implementation

computation

```
(\_.  
  perform ask () + perform ask ()  
)
```

perform an effect

Algebraic effects 101

effect

```
effect read {  
  ask : () -> int  
}
```

operation

effect handler

```
handler {  
  ask -> \x.\k. k 1  
}
```

implementation

computation

```
(\_.  
  perform ask () + perform ask ()  
) // 2
```

perform an effect

Operational semantics of untyped algebraic effects

- ① (*app*) $(\lambda x. e) v \longrightarrow e[x := v]$
- ② (*handler*) $\text{handler } h f \longrightarrow \text{handle } h (f ())$
- ③ (*return*) $\text{handle } h v \longrightarrow v$
- ④ (*perform*) $\text{handle } h E[\text{perform } op v] \longrightarrow f v (\lambda x. \text{handle } h E[x])$
iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

Operational semantics of untyped algebraic effects

$$\textcircled{1} \quad (\textit{app}) \quad (\lambda x. e) \ v \quad \longrightarrow \quad e[x := v]$$

$$\textcircled{2} \quad (\textit{handler}) \quad \textit{handler } h \ f \quad \longrightarrow \quad \textit{handle } h \ (f \ ())$$

$$\textcircled{3} \quad (\textit{return}) \quad \textit{handle } h \ v \quad \longrightarrow \quad v$$

$$\textcircled{4} \quad (\textit{perform}) \quad \textit{handle } h \ E[\textit{perform } op \ v] \longrightarrow \begin{array}{l} f \ v \ (\lambda x. \textit{handle } h \ E[x]) \\ \text{iff } op \notin \text{bop}(E) \wedge (op \mapsto f) \in h \end{array}$$

Operational semantics of untyped algebraic effects

- ① (*app*) $(\lambda x. e) v \longrightarrow e[x := v]$
- ② (*handler*) $\text{handler } h f \longrightarrow \text{handle } h (f ())$
- ③ (*return*) $\text{handle } h v \longrightarrow v$
- ④ (*perform*) $\text{handle } h E[\text{perform } op v] \longrightarrow f v (\lambda x. \text{handle } h E[x])$
iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

Operational semantics of untyped algebraic effects

$$\textcircled{1} \quad (\textit{app}) \quad (\lambda x. e) \, v \longrightarrow e[x := v]$$

a unit-taking function as a computation

$$\textcircled{2} \quad (\textit{handler}) \quad \textit{handler } h \, f \longrightarrow \textit{handle } h \, (f \, ())$$

$$\textcircled{3} \quad (\textit{return}) \quad \textit{handle } h \, v \longrightarrow v$$

$$\textcircled{4} \quad (\textit{perform}) \quad \textit{handle } h \, E[\textit{perform } op \, v] \longrightarrow f \, v \, (\lambda x. \textit{handle } h \, E[x])$$

iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

Operational semantics of untyped algebraic effects

$$\textcircled{1} \quad (\textit{app}) \quad (\lambda x. e) \ v \quad \longrightarrow \quad e[x := v]$$

a unit-taking function as a computation

$$\textcircled{2} \quad (\textit{handler}) \quad \textit{handler } h \ f \quad \longrightarrow \quad \textit{handle } h \ (f \ ())$$

$$\textcircled{3} \quad (\textit{return}) \quad \textit{handle } h \ v \quad \longrightarrow \quad v$$

$$\textcircled{4} \quad (\textit{perform}) \quad \textit{handle } h \ E[\textit{perform } op \ v] \quad \longrightarrow \quad f \ v \ (\lambda x. \textit{handle } h \ E[x])$$

iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

Operational semantics of untyped algebraic effects

$$\textcircled{1} \quad (\textit{app}) \quad (\lambda x. e) \ v \quad \longrightarrow \quad e[x := v]$$

a unit-taking function as a computation

$$\textcircled{2} \quad (\textit{handler}) \quad \textit{handler } h \ f \quad \longrightarrow \quad \textit{handle } h \ (f \ ())$$

$$\textcircled{3} \quad (\textit{return}) \quad \textit{handle } h \ v \quad \longrightarrow \quad v$$

$$\textcircled{4} \quad (\textit{perform}) \quad \textit{handle } h \ E[\textit{perform } op \ v] \longrightarrow f \ v \ (\lambda x. \textit{handle } h \ E[x]) \\ \text{iff } op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$$

Operational semantics of untyped algebraic effects

$$\textcircled{1} \quad (\textit{app}) \quad (\lambda x. e) \ v \quad \longrightarrow \quad e[x := v]$$

a unit-taking function as a computation

$$\textcircled{2} \quad (\textit{handler}) \quad \textit{handler } h \ f \quad \longrightarrow \quad \textit{handle } h \ (f \ ())$$

$$\textcircled{3} \quad (\textit{return}) \quad \textit{handle } h \ v \quad \longrightarrow \quad v$$

evaluation context

$$\textcircled{4} \quad (\textit{perform}) \quad \textit{handle } h \ \boxed{E}[\textit{perform } op \ v] \quad \longrightarrow \quad f \ v \ (\lambda x. \textit{handle } h \ E[x])$$

iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

Operational semantics of untyped algebraic effects

$$\textcircled{1} \quad (\textit{app}) \quad (\lambda x. e) \ v \quad \longrightarrow \quad e[x := v]$$

a unit-taking function as a computation

$$\textcircled{2} \quad (\textit{handler}) \quad \textit{handler } h \ f \quad \longrightarrow \quad \textit{handle } h \ (f \ ())$$

$$\textcircled{3} \quad (\textit{return}) \quad \textit{handle } h \ v \quad \longrightarrow \quad v$$

evaluation context

$$\textcircled{4} \quad (\textit{perform}) \quad \textit{handle } h \ \boxed{E}[\textit{perform } op \ v] \quad \longrightarrow \quad f \ v \ (\lambda x. \textit{handle } h \ E[x])$$

iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

h is the
innermost
handler

Operational semantics of untyped algebraic effects

$$\textcircled{1} \quad (\textit{app}) \quad (\lambda x. e) \ v \quad \longrightarrow \quad e[x := v]$$

a unit-taking function as a computation

$$\textcircled{2} \quad (\textit{handler}) \quad \textit{handler } h \ f \quad \longrightarrow \quad \textit{handle } h \ (f \ ())$$

$$\textcircled{3} \quad (\textit{return}) \quad \textit{handle } h \ v \quad \longrightarrow \quad v$$

evaluation context

$$\textcircled{4} \quad (\textit{perform}) \quad \textit{handle } h \ \boxed{E}[\textit{perform } op \ v] \quad \longrightarrow \quad f \ v \ (\lambda x. \textit{handle } h \ E[x])$$

iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

h is the innermost handler

get the operation implementation f

Operational semantics of untyped algebraic effects

$$\textcircled{1} \quad (\textit{app}) \quad (\lambda x. e) \ v \quad \longrightarrow \quad e[x := v]$$

a unit-taking function as a computation

$$\textcircled{2} \quad (\textit{handler}) \quad \textit{handler } h \ f \quad \longrightarrow \quad \textit{handle } h \ (f \ ())$$

$$\textcircled{3} \quad (\textit{return}) \quad \textit{handle } h \ v \quad \longrightarrow \quad v$$

evaluation context

operation argument

$$\textcircled{4} \quad (\textit{perform}) \quad \textit{handle } h \ \boxed{E}[\textit{perform } op \ v] \quad \longrightarrow \quad f \ v \ (\lambda x. \textit{handle } h \ E[x])$$

iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

h is the innermost handler

get the operation implementation f

Operational semantics of untyped algebraic effects

$$\textcircled{1} \quad (\textit{app}) \quad (\lambda x. e) \ v \quad \longrightarrow \quad e[x := v]$$

a unit-taking function as a computation

$$\textcircled{2} \quad (\textit{handler}) \quad \textit{handler} \ h \ f \quad \longrightarrow \quad \textit{handle} \ h \ (f \ ())$$

$$\textcircled{3} \quad (\textit{return}) \quad \textit{handle} \ h \ v \quad \longrightarrow \quad v$$

$$\textcircled{4} \quad (\textit{perform}) \quad \textit{handle} \ h \ \boxed{E}[\textit{perform} \ op \ v] \quad \longrightarrow \quad f \ v \ \boxed{(\lambda x. \textit{handle} \ h \ E[x])}$$

iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

h is the innermost handler

get the operation implementation f

The problem: compiling effect handlers efficiently

$$\begin{aligned} (\textit{perform}) \quad \text{handle } h \ E[\text{perform } op \ v] &\longrightarrow f \ v \ (\lambda x. \text{handle } h \ E[x]) \\ &\text{iff } op \notin \text{bop}(E) \wedge (op \mapsto f) \in h \end{aligned}$$

The problem: compiling effect handlers efficiently

$$\begin{aligned} (\textit{perform}) \quad \text{handle } h \ E[\text{perform } op \ v] &\longrightarrow f \ v \ (\lambda x. \text{handle } h \ E[x]) \\ &\text{iff } op \notin \text{bop}(E) \wedge (op \mapsto f) \in h \end{aligned}$$

Two potentially **expensive** runtime operations:

The problem: compiling effect handlers efficiently

evaluation context

$$\begin{aligned} (\textit{perform}) \quad \text{handle } h \boxed{E}[\text{perform } op \ v] &\longrightarrow f \ v \ (\lambda x. \text{handle } h \ E[x]) \\ &\text{iff } op \notin \text{bop}(E) \wedge (op \mapsto f) \in h \end{aligned}$$

Two potentially **expensive** runtime operations:

The problem: compiling effect handlers efficiently

evaluation context

$$\begin{aligned} (\textit{perform}) \quad \text{handle } h \boxed{E}[\text{perform } op \ v] &\longrightarrow f \ v \ (\lambda x. \text{handle } h \ E[x]) \\ &\text{iff } op \notin \text{bop}(E) \wedge (op \mapsto f) \in h \\ &\text{h is the innermost handler} \end{aligned}$$

Two potentially **expensive** runtime operations:

The problem: compiling effect handlers efficiently

evaluation context

$$\begin{aligned} (\textit{perform}) \quad \text{handle } h \boxed{E}[\text{perform } op \ v] &\longrightarrow f \ v \ (\lambda x. \text{handle } h \ E[x]) \\ &\text{iff } op \notin \text{bop}(E) \wedge (op \mapsto f) \in h \\ &\text{h is the innermost handler} \end{aligned}$$

Two potentially **expensive** runtime operations:

1. **Searching**

a *linear* search through the current evaluation context to find the innermost handler for op

The problem: compiling effect handlers efficiently

$$\begin{array}{c} \text{evaluation context} \\ (perform) \quad \text{handle } h \boxed{E[\text{perform } op \ v]} \longrightarrow f \ v \boxed{(\lambda x. \text{handle } h \ E[x])} \\ \text{resumption} \\ \text{iff } op \notin \text{bop}(E) \wedge (op \mapsto f) \in h \\ h \text{ is the innermost handler} \end{array}$$

Two potentially **expensive** runtime operations:

1. **Searching**

a *linear* search through the current evaluation context to find the innermost handler for op

The problem: compiling effect handlers efficiently

$$\begin{array}{c} \text{evaluation context} \\ (perform) \quad \text{handle } h \boxed{E[\text{perform } op \ v]} \longrightarrow f \ v \boxed{(\lambda x. \text{handle } h \ E[x])} \\ \text{resumption} \\ \text{iff } op \notin \text{bop}(E) \wedge (op \mapsto f) \in h \\ h \text{ is the innermost handler} \end{array}$$

Two potentially **expensive** runtime operations:

1. **Searching**

a *linear* search through the current evaluation context to find the innermost handler for op

2. **Capturing**

capture the evaluation context (i.e., stacks and registers) up to the found handler, and create a resumption function

The problem: compiling effect handlers efficiently

$$\begin{array}{c} \text{evaluation context} \\ (perform) \quad \text{handle } h \boxed{E[\text{perform } op \ v]} \longrightarrow f \ v \boxed{(\lambda x. \text{handle } h \ E[x])} \\ \text{resumption} \\ \text{iff } op \notin \text{bop}(E) \wedge (op \mapsto f) \in h \\ \text{h is the innermost handler} \end{array}$$

Two potentially **expensive** runtime operations:

1. **Searching**

a *linear* search through the current evaluation context to find the innermost handler for op

2. **Capturing**

capture the evaluation context (i.e., stacks and registers) up to the found handler, and create a resumption function

This work:

Multi-prompt
delimited control

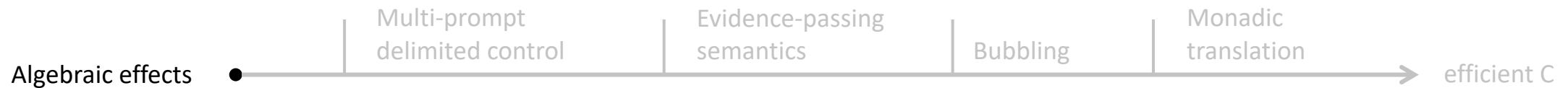
Evidence-passing
semantics

Monadic
translation

efficient C

Reader, formally

```
handler {  
  ask -> \x.\k. k 1  
}  
(\_.  
  perform ask () + perform ask ()  
)
```



Reader, formally

```
handler {  
  ask -> \x.\k. k 1  
}  
(\_.  
  perform ask () + perform ask ()  
)
```

```
f = \x.\k. k 1  
h1 = ask -> f
```



Reader, formally

handler

```
h1
(\_.
  perform ask () + perform ask ()
)
```

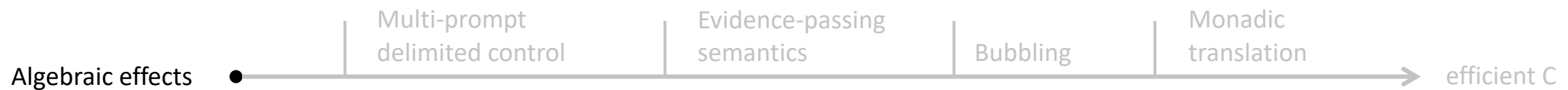
```
f = \x.\k. k 1
h1 = ask -> f
```



Reader, formally

```
handler h1 (\_. perform ask () + perform ask ())
```

```
f = \x.\k. k 1  
h1 = ask -> f
```



Reader, formally

handler h1 (_. **perform** ask () + **perform** ask ())
 $\mapsto$ (*handler*) handler *h* *f* $\longrightarrow$ handle *h* (*f* ())

```
f = \x.\k. k 1
h1 = ask -> f
```



Reader, formally

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$\mapsto$ (*handler*) handler h f $\longrightarrow$ handle h (f ())

$\mapsto$ (*app*) $(\lambda x. e)$ v $\longrightarrow$ $e[x:=v]$

$f = \backslash x. \backslash k. k$ **1**
 $h1 = ask \rightarrow f$



Reader, formally

handler h1 (_. **perform** ask () + **perform** ask ())

$\mapsto$ (*handler*) handler *h f* $\longrightarrow$ handle *h* (*f* ())

$\mapsto$ (*app*) ($\lambda x. e$) *v* $\longrightarrow$ *e*[*x*:=*v*]

handle h1 (**perform** ask () + **perform** ask ())

$$\begin{aligned} f &= \lambda x. \lambda k. k \text{ 1} \\ h1 &= ask \rightarrow f \end{aligned}$$



Reader, formally

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

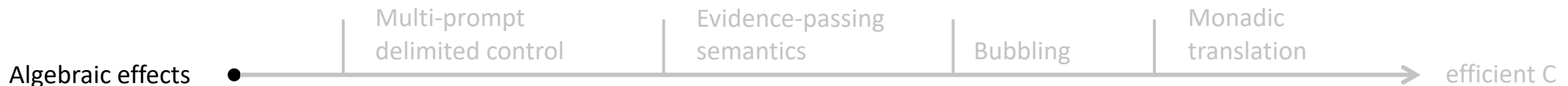
$\mapsto$ (*handler*) handler h f $\longrightarrow$ handle h (f ())

$\mapsto$ (*app*) $(\lambda x. e)$ v $\longrightarrow$ $e[x:=v]$

handle h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (*perform*) handle h $E[\text{perform } op \ v]$ $\longrightarrow$ $f \ v (\lambda x. \text{handle } h \ E[x])$
iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

$f = \backslash x. \backslash k. k \ \mathbf{1}$
 $h1 = ask \rightarrow f$



Reader, formally

$$f = \lambda x. \lambda k. k \text{ } 1$$

$$h1 = ask \rightarrow f$$

handler h1 ($\lambda _.$ **perform** ask () + **perform** ask ())

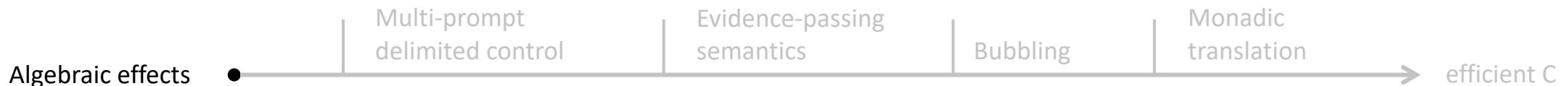
$\mapsto$ (*handler*) handler h $f \longrightarrow \text{handle } h (f ())$

$\mapsto$ (*app*) $(\lambda x. e) v \longrightarrow e[x := v]$

handle h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (*perform*) $\text{handle } h E[\text{perform } op \ v] \longrightarrow f \ v (\lambda x. \text{handle } h E[x])$
iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

$f \ () \ (\lambda x. \text{handle } h1 \ (x + \text{perform } ask \ ()))$



Reader, formally

$$f = \lambda x. \lambda k. k \text{ } 1$$

$$h1 = ask \rightarrow f$$

handler h1 ($\lambda _.$ **perform** ask () + **perform** ask ())

$\mapsto$ (*handler*) handler $h f \longrightarrow \text{handle } h (f ())$

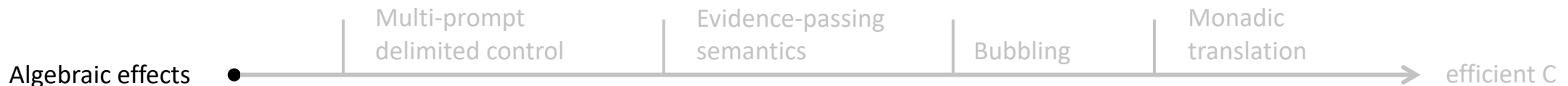
$\mapsto$ (*app*) $(\lambda x. e) v \longrightarrow e[x:=v]$

handle h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (*perform*) $\text{handle } h E[\text{perform } op \ v] \longrightarrow f \ v (\lambda x. \text{handle } h E[x])$
iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

$f \ () \ (\lambda x. \text{handle } h1 \ (x + \text{perform } ask \ ()))$

$\mapsto (app)^*$



Reader, formally

$$f = \lambda x. \lambda k. k \text{ } \mathbf{1}$$

$$h1 = ask \rightarrow f$$

handler h1 ($\lambda _.$ **perform** ask () + **perform** ask ())

$\mapsto$ (*handler*) handler $h f \longrightarrow \text{handle } h (f ())$

$\mapsto$ (*app*) ($\lambda x. e$) $v \longrightarrow e[x := v]$

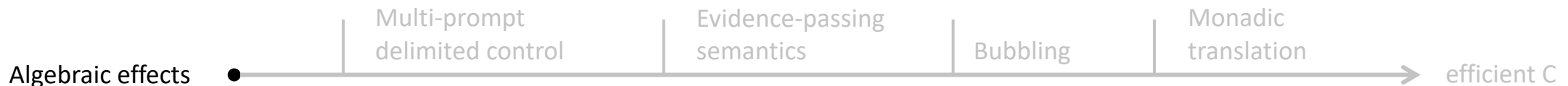
handle h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (*perform*) $\text{handle } h E[\text{perform } op \ v] \longrightarrow f \ v (\lambda x. \text{handle } h E[x])$
iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

$f \ () \ (\lambda x. \text{handle } h1 \ (x + \text{perform } ask \ ()))$

$\mapsto$ (*app*)^{*}

$(\lambda x. \text{handle } h1 \ (x + \text{perform } ask \ ())) \ \mathbf{1}$



Reader, formally

$$f = \lambda x. \lambda k. k \text{ } \mathbf{1}$$

$$h1 = ask \rightarrow f$$

handler h1 (_ . **perform** ask () + **perform** ask ())

$\mapsto$ (*handler*) handler $h f \longrightarrow \text{handle } h (f ())$

$\mapsto$ (*app*) $(\lambda x. e) v \longrightarrow e[x := v]$

handle h1 (**perform** ask () + **perform** ask ())

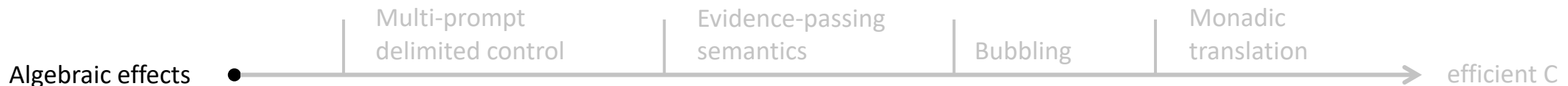
$\mapsto$ (*perform*) $\text{handle } h E[\text{perform } op \ v] \longrightarrow f \ v (\lambda x. \text{handle } h E[x])$
iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

$f \ () \ (\lambda x. \text{handle } h1 \ (x + \text{perform } ask \ ()))$

$\mapsto$ (*app*)^{*}

$(\lambda x. \text{handle } h1 \ (x + \text{perform } ask \ ())) \ \mathbf{1}$

$\mapsto$ (*app*)



Reader, formally

$$f = \lambda x. \lambda k. k \text{ } \mathbf{1}$$

$$h1 = ask \rightarrow f$$

handler h1 ($\lambda _.$ **perform** ask () + **perform** ask ())

$\mapsto$ (*handler*) handler $h f \longrightarrow \text{handle } h (f ())$

$\mapsto$ (*app*) $(\lambda x. e) v \longrightarrow e[x := v]$

handle h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (*perform*) $\text{handle } h E[\text{perform } op \ v] \longrightarrow f \ v (\lambda x. \text{handle } h E[x])$
iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

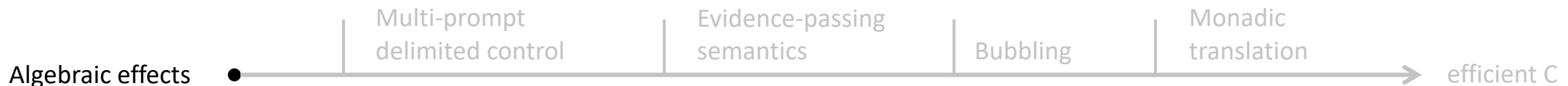
$f \ () \ (\lambda x. \text{handle } h1 \ (x + \text{perform } ask \ ()))$

$\mapsto$ (*app*)^{*}

$(\lambda x. \text{handle } h1 \ (x + \text{perform } ask \ ())) \ \mathbf{1}$

$\mapsto$ (*app*)

handle h1 (**1** + **perform** ask ())



Reader, formally

$$f = \lambda x. \lambda k. k \text{ } \mathbf{1}$$

$$h1 = ask \rightarrow f$$

handler h1 ($\lambda _.$ **perform** ask () + **perform** ask ())

$\mapsto$ (*handler*) handler $h f \longrightarrow \text{handle } h (f ())$

$\mapsto$ (*app*) ($\lambda x. e$) $v \longrightarrow e[x := v]$

handle h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (*perform*) $\text{handle } h E[\text{perform } op \ v] \longrightarrow f \ v (\lambda x. \text{handle } h E[x])$
iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

$f \ () \ (\lambda x. \text{handle } h1 \ (x + \text{perform } ask \ ()))$

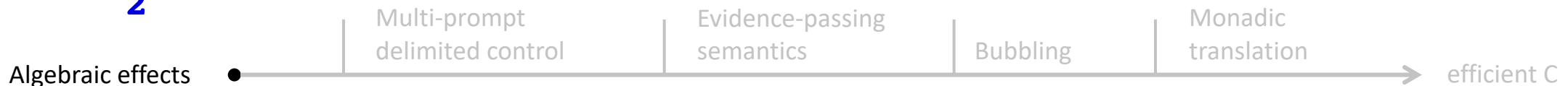
$\mapsto$ (*app*)^{*}

($\lambda x. \text{handle } h1 \ (x + \text{perform } ask \ ()) \ \mathbf{1}$)

$\mapsto$ (*app*)

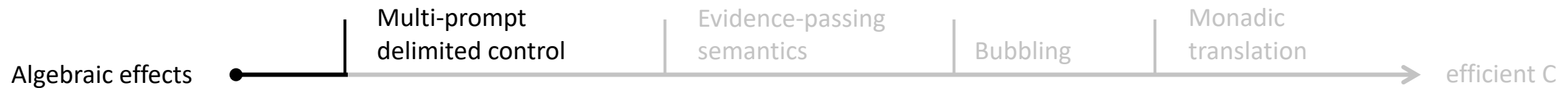
handle h1 (**1** + **perform** ask ())

$\mapsto^*$ **2**



Multi-prompt semantics

separating searching from capturing



Multi-prompt semantics

separating searching from capturing

```
handler h1 (\_. perform ask () + perform ask ())
```

$\mapsto$ (*handler*) (*app*)

```
handle h1 (perform ask () + perform ask ())
```

$\mapsto$ (*perform*)

```
f () (\x. handle h1 (x + perform ask ()))
```

$\mapsto$ (*app*)^{*}

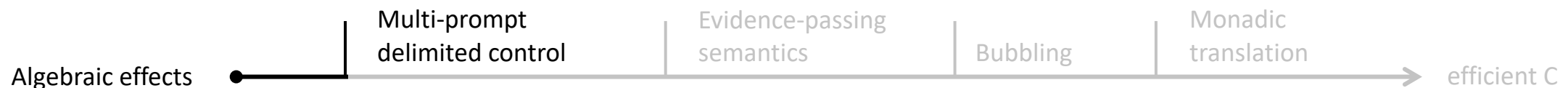
```
(\x. handle h1 (x + perform ask ())) 1
```

$\mapsto$ (*app*)

```
handle h1 (1 + perform ask ())
```

$\mapsto^*$ **2**

```
f = \x.\k. k 1  
h1 = ask -> f
```



Multi-prompt semantics

separating searching from capturing

```
handler h1 (\_. perform ask () + perform ask ())
```

$\mapsto$ (*handler*) (*app*)

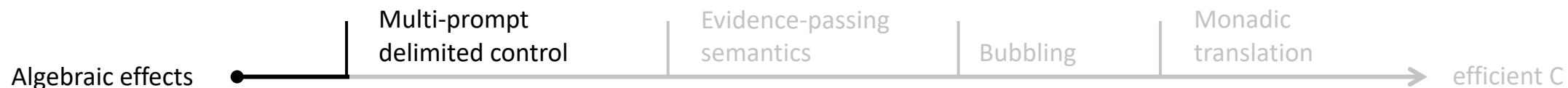
```
handle h1 (perform ask () + perform ask ())
```

$\mapsto$ (*perform*)

```
f () (\x. handle h1 (x + perform ask ()))
```

$\mapsto^*$ **2**

```
f = \x.\k. k 1  
h1 = ask -> f
```



Multi-prompt semantics

separating searching from capturing

```
handler h1 (\_. perform ask () + perform ask ())
```

$\mapsto$ (*handler*) (*app*)

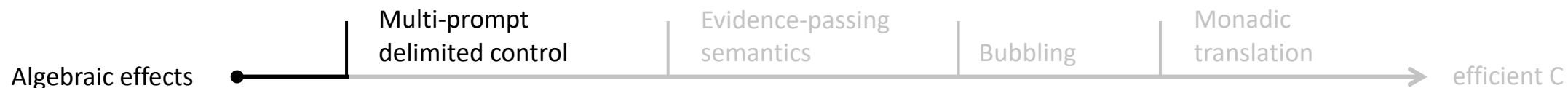
```
handle h1 (perform ask () + perform ask ())
```

$\mapsto$ (*perform*)

```
f () (\x. handle h1 (x + perform ask ()))
```

$\mapsto^*$ **2**

```
f = \x.\k. k 1  
h1 = ask -> f
```



Multi-prompt semantics

separating searching from capturing

```
handler h1 (\_. perform ask () + perform ask ())
```

$\mapsto$ (*handler*) (*app*)

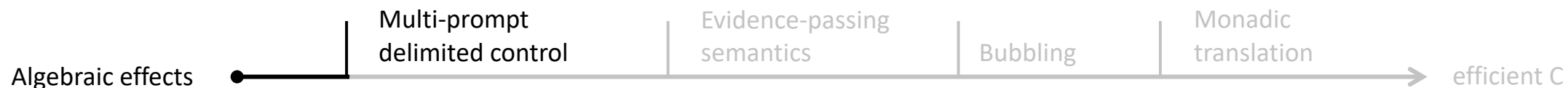
```
prompt m1 h1 (perform ask () + perform ask ())
```

$\mapsto$ (*perform*)

```
f () (\x. prompt m1 h1 (x + perform ask ()))
```

$\mapsto^*$ **2**

```
f = \x.\k. k 1  
h1 = ask -> f
```



Multi-prompt semantics

separating searching from capturing

```
handler h1 (\_. perform ask () + perform ask ())
```

$\mapsto$ (*handler*) (*app*)

```
prompt m1 h1 (perform ask () + perform ask ())
```

m1: a unique marker
identifying handlers

$\mapsto$ (*perform*)

```
f () (\x. prompt m1 h1 (x + perform ask ()))
```

$\mapsto^*$ 2



Multi-prompt semantics

separating searching from capturing

```
handler h1 (\_. perform ask () + perform ask ())
```

$\mapsto$ (*handler*) (*app*)

```
prompt m1 h1 (perform ask () + perform ask ())
```

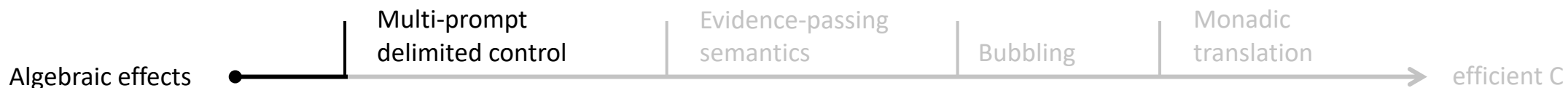
$\mapsto$ (*perform*)

```
f () (\x. prompt m1 h1 (x + perform ask ()))
```

$\mapsto^*$ **2**

```
f = \x.\k. k 1  
h1 = ask -> f
```

m1: a unique marker
identifying handlers



Multi-prompt semantics

separating searching from capturing

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$\mapsto$ (*handler*) (*app*)

prompt m1 h1 (**perform** ask () + **perform** ask ())

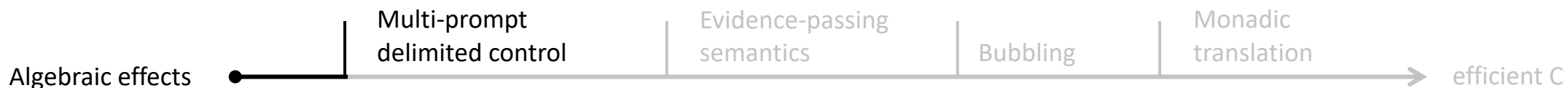
$\mapsto$ (*perform*) iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

f () ($\backslash x.$ **prompt** m1 h1 (x + **perform** ask ()))

$\mapsto^*$ 2

$f = \backslash x. \backslash k. k \text{ 1}$
 $h1 = ask \rightarrow f$

m1: a unique marker
identifying handlers



Multi-prompt semantics

separating searching from capturing

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$\mapsto$ (handler) (app)

prompt m1 h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (perform) iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

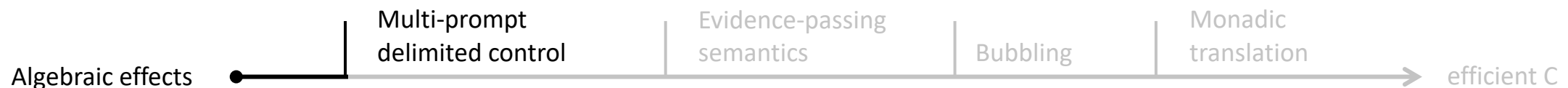
prompt m1 h1 (**yield** m1 ($\backslash k.$ f () k) + **perform** ask ())

$f = \backslash x.\backslash k. k$ **1**
 $h1 = \text{ask} \rightarrow f$

m1: a unique marker
identifying handlers

f () ($\backslash x.$ **prompt** m1 h1 (x + **perform** ask ()))

$\mapsto^*$ **2**



Multi-prompt semantics

separating searching from capturing

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$f = \backslash x.\backslash k. k \text{ 1}$
 $h1 = ask \rightarrow f$

$\mapsto$ (handler) (app)

prompt m1 h1 (**perform** ask () + **perform** ask ())

m1: a unique marker
identifying handlers

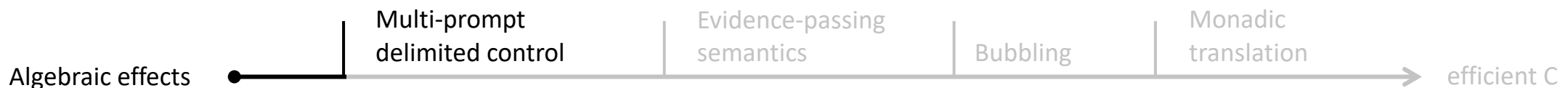
$\mapsto$ (perform) iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

prompt m1 h1 (**yield** m1 ($\backslash k. f () k$) + **perform** ask ())

yielding to a handler
identified by m1

$f () (\backslash x. \text{prompt } m1 \text{ h1 } (x + \text{perform } ask ()))$

$\mapsto^* \text{ 2}$



Multi-prompt semantics

separating searching from capturing

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$f = \backslash x.\backslash k. k \text{ 1}$
 $h1 = ask \rightarrow f$

$\mapsto$ (handler) (app)

prompt m1 h1 (**perform** ask () + **perform** ask ())

m1: a unique marker
identifying handlers

$\mapsto$ (perform) iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

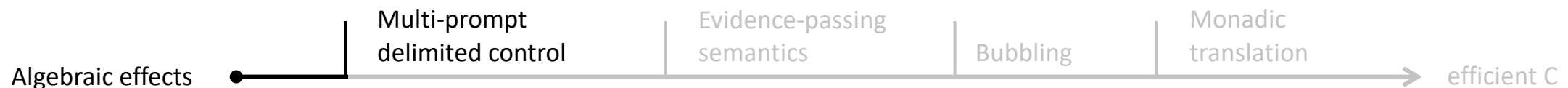
prompt m1 h1 (**yield** m1 ($\backslash k.$ **f** () k) + **perform** ask ())

operation implementation partially
applied to operation argument

yielding to a handler
identified by m1

f () ($\backslash x.$ **prompt** m1 h1 (x + **perform** ask ()))

$\mapsto^* \text{ 2}$



Multi-prompt semantics

separating searching from capturing

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$\mapsto$ (handler) (app)

prompt m1 h1 (**perform** ask () + **perform** ask ())

m1: a unique marker
identifying handlers

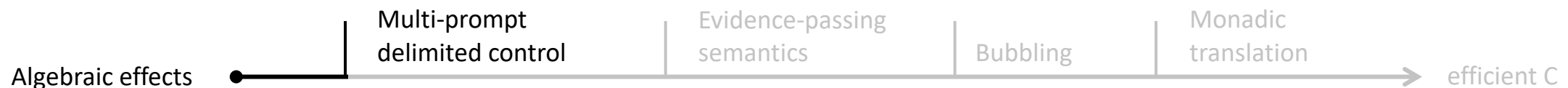
$\mapsto$ (perform) iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

prompt m1 h1 (**yield** m1 ($\backslash k.$ f () k) + **perform** ask ())

yielding to a handler
identified by m1

f () ($\backslash x.$ **prompt** m1 h1 (x + **perform** ask ()))

$\mapsto^*$ 2



Multi-prompt semantics

separating searching from capturing

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$f = \backslash x. \backslash k. k \text{ 1}$
 $h1 = ask \rightarrow f$

$\mapsto$ (*handler*) (*app*)

prompt m1 h1 (**perform** ask () + **perform** ask ())

m1: a unique marker
identifying handlers

$\mapsto$ (*perform*) iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

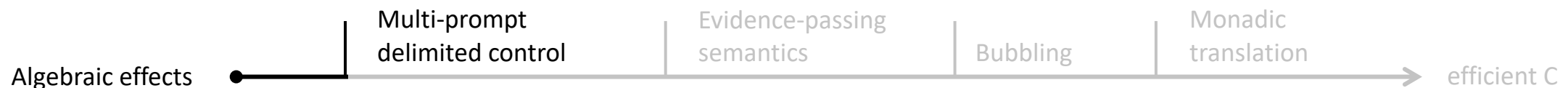
prompt m1 h1 (**yield** m1 ($\backslash k. f () k$) + **perform** ask ())

yielding to a handler
identified by m1

$\mapsto$ (*prompt*)

$f () (\backslash x. \text{prompt m1 h1 } (x + \text{perform ask } ()))$

$\mapsto^* \text{ 2}$



Multi-prompt semantics

separating searching from capturing

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$\mapsto$ (*handler*) (*app*)

prompt m1 h1 (**perform** ask () + **perform** ask ())

m1: a unique marker
identifying handlers

$\mapsto$ (*perform*) iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

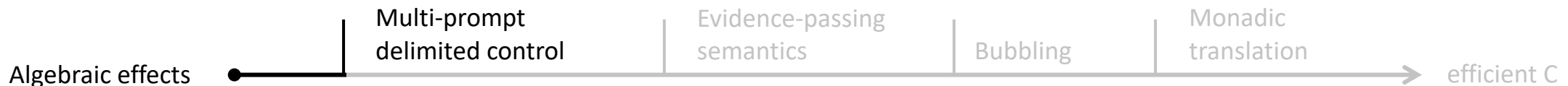
prompt m1 h1 (**yield** m1 ($\backslash k.$ f () k) + **perform** ask ())

yielding to a handler
identified by m1

$\mapsto$ (*prompt*)

f () ($\backslash x.$ **prompt** m1 h1 (x + **perform** ask ()))

$\mapsto^*$ 2



Multi-prompt semantics

separating searching from capturing

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$\mapsto$ (*handler*) (*app*)

prompt m1 h1 (**perform** ask () + **perform** ask ())

m1: a unique marker
identifying handlers

$\mapsto$ (*perform*) iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$

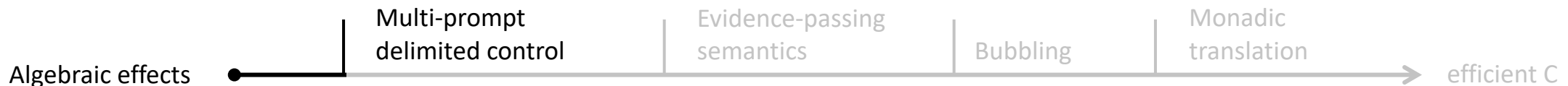
prompt m1 h1 (**yield** m1 ($\backslash k.$ f () k) + **perform** ask ())

yielding to a handler
identified by m1

$\mapsto$ (*prompt*)

f () ($\backslash x.$ **prompt** m1 h1 (x + **perform** ask ()))

$\mapsto^*$ **2**



Multi-prompt semantics

separating searching from capturing

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$f = \backslash x. \backslash k. k \text{ 1}$
 $h1 = ask \rightarrow f$

$\mapsto$ (*handler*) (*app*)

prompt m1 h1 (**perform** ask () + **perform** ask ())

m1: a unique marker
identifying handlers

$\mapsto$ (*perform*) iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$ searching

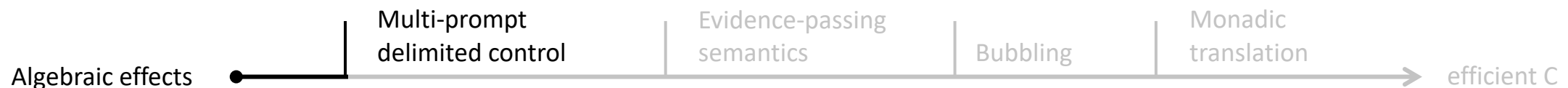
prompt m1 h1 (**yield** m1 ($\backslash k. f () k$) + **perform** ask ())

yielding to a handler
identified by m1

$\mapsto$ (*prompt*)

$f () (\backslash x. \text{prompt } m1 \ h1 \ (x + \text{perform } ask ()))$

$\mapsto^* \text{ 2}$



Multi-prompt semantics

separating searching from capturing

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$f = \backslash x. \backslash k. k \text{ 1}$
 $h1 = ask \rightarrow f$

$\mapsto$ (*handler*) (*app*)

prompt m1 h1 (**perform** ask () + **perform** ask ())

m1: a unique marker
identifying handlers

$\mapsto$ (*perform*) iff $op \notin \text{bop}(E) \wedge (op \mapsto f) \in h$ searching

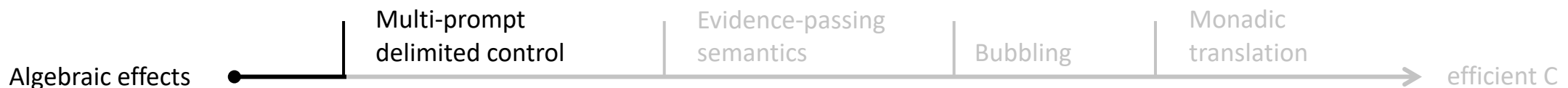
prompt m1 h1 (**yield** m1 ($\backslash k. f () k$) + **perform** ask ())

yielding to a handler
identified by m1

$\mapsto$ (*prompt*) capturing

$f () (\backslash x. \text{prompt } m1 \ h1 \ (x + \text{perform } ask ()))$

$\mapsto^* \text{ 2}$



Evidence-passing semantics

make performs local: push down the current handlers as an evidence vector

```
handler h1 (\_. perform ask () + perform ask ())
```

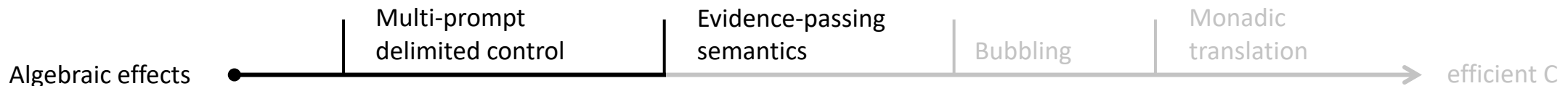
$f = \lambda x. \lambda k. k \text{ 1}$
 $h1 = ask \rightarrow f$

$\mapsto (handler) (app)$

```
prompt m1 h1 (perform ask () + perform ask ())
```

$\mapsto (perform) \text{ iff } op \notin \text{bop}(E) \wedge (op \mapsto f) \in h \quad \text{searching}$

```
prompt m1 h1 (yield m1 (\k. f () k) + perform ask ())
```



Evidence-passing semantics

make performs local: push down the current handlers as an evidence vector

```
f = \x.\k. k 1
h1 = ask -> f
```

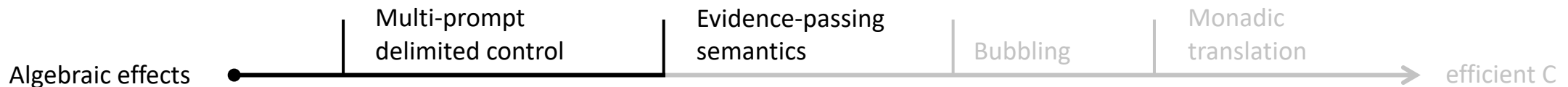
```
handler h1 (\_. perform ask () + perform ask ())
```

$\mapsto$ (*handler*) (*app*)

```
prompt m1 h1 (perform ask () + perform ask ())
```

$\mapsto$ (*perform*)

```
prompt m1 h1 (yield m1 (\k. f () k) + perform ask ())
```



Evidence-passing semantics

make performs local: push down the current handlers as an evidence vector

```
f = \x.\k. k 1
h1 = ask -> f
```

$\langle\langle\rangle\rangle$

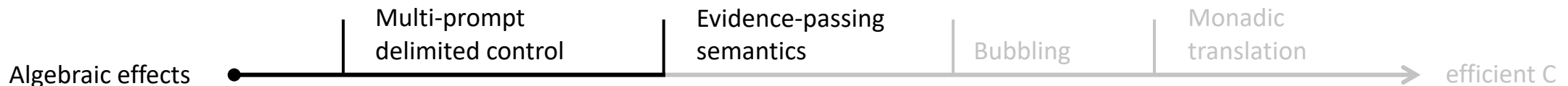
```
handler h1 (\_. perform ask () + perform ask ())
```

$\mapsto (handler) (app)$

```
prompt m1 h1 (perform ask () + perform ask ())
```

$\mapsto (perform)$

```
prompt m1 h1 (yield m1 (\k. f () k) + perform ask ())
```



Evidence-passing semantics

make performs local: push down the current handlers as an evidence vector

```
f = \x.\k. k 1
h1 = ask -> f
```

$\langle\langle\rangle\rangle$

```
handler h1 (\_. perform ask () + perform ask ())
```

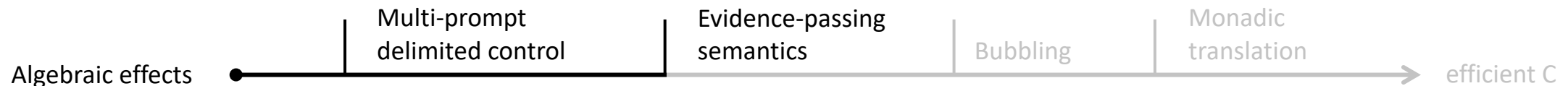
$\mapsto (handler) (app)$

$\langle\langle\rangle\rangle$

```
prompt m1 h1 (perform ask () + perform ask ())
```

$\mapsto (perform)$

```
prompt m1 h1 (yield m1 (\k. f () k) + perform ask ())
```



Evidence-passing semantics

make performs local: push down the current handlers as an evidence vector

```
f = \x.\k. k 1
h1 = ask -> f
```

$\langle\langle\rangle\rangle$

```
handler h1 (\_. perform ask () + perform ask ())
```

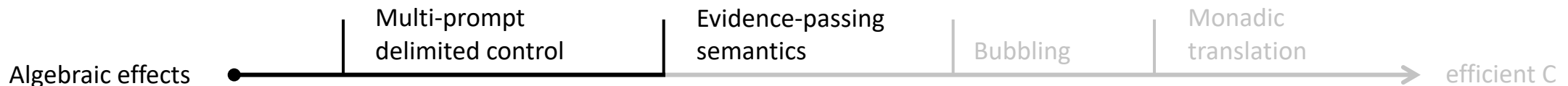
$\mapsto (handler) (app)$

$\langle\langle\rangle\rangle$ $\langle\langle read: (m1, h1) \rangle\rangle$

```
prompt m1 h1 (perform ask () + perform ask ())
```

$\mapsto (perform)$

```
prompt m1 h1 (yield m1 (\k. f () k) + perform ask ())
```



Evidence-passing semantics

make performs local: push down the current handlers as an evidence vector

$$f = \lambda x. \lambda k. k \text{ 1}$$
$$h1 = ask \rightarrow f$$

$\langle\langle\rangle\rangle$

$\text{handler } h1 \ (\lambda _ . \text{perform } ask \ () + \text{perform } ask \ ())$

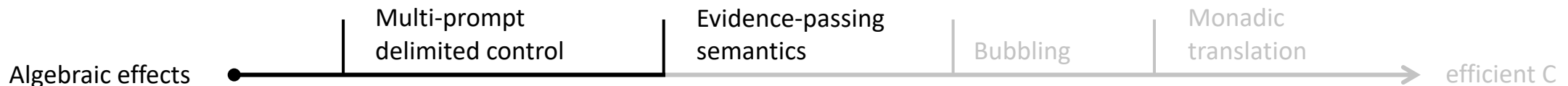
$\mapsto (handler) (app)$

$\langle\langle\rangle\rangle$ $\langle\langle read: (m1, h1) \rangle\rangle$

$\text{prompt } m1 \ h1 \ (\text{perform } ask \ () + \text{perform } ask \ ())$

$\mapsto (perform) \ \langle\langle read: (m1, h1) \rangle\rangle$

$\text{prompt } m1 \ h1 \ (\text{yield } m1 \ (\lambda k. f \ () \ k) + \text{perform } ask \ ())$



Evidence-passing semantics

make performs local: push down the current handlers as an evidence vector

```
f = \x.\k. k 1
h1 = ask -> f
```

$\langle\langle\rangle\rangle$

```
handler h1 (\_. perform ask () + perform ask ())
```

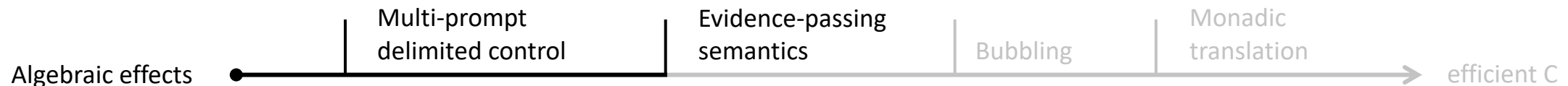
$\mapsto (handler) (app)$

$\langle\langle\rangle\rangle$ $\langle\langle read: (m1, h1) \rangle\rangle$

```
prompt m1 h1 (perform ask () + perform ask ())
```

$\mapsto (perform) \langle\langle read: (m1, h1) \rangle\rangle.read = (m1, h1)$

```
prompt m1 h1 (yield m1 (\k. f () k) + perform ask ())
```



Evidence-passing semantics

make performs local: push down the current handlers as an evidence vector

```
f = \x.\k. k 1
h1 = ask -> f
```

$\langle\langle\rangle\rangle$

handler h1 (_. **perform** ask () + **perform** ask ())

$\mapsto$ (handler) (app)

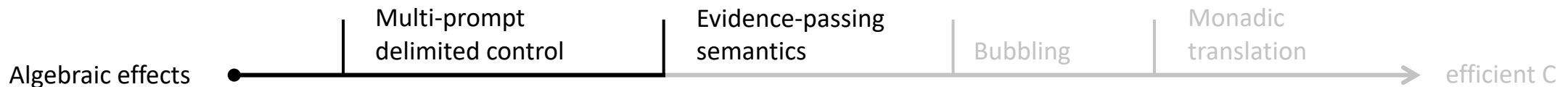
$\langle\langle\rangle\rangle$ $\langle\langle \text{read}: (m1, h1) \rangle\rangle$

prompt m1 h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (perform) $\langle\langle \text{read}: (m1, h1) \rangle\rangle$.read = (m1, h1)

$\langle\langle\rangle\rangle$ $\langle\langle \text{read}: (m1, h1) \rangle\rangle$

prompt m1 h1 (**yield** m1 (\k. f () k) + **perform** ask ())



Evidence-passing semantics

make performs local: push down the current handlers as an evidence vector

```
f = \x.\k. k 1
h1 = ask -> f
```

$\langle\langle\rangle\rangle$

handler h1 (_. **perform** ask () + **perform** ask ())

$\mapsto$ (handler) (app)

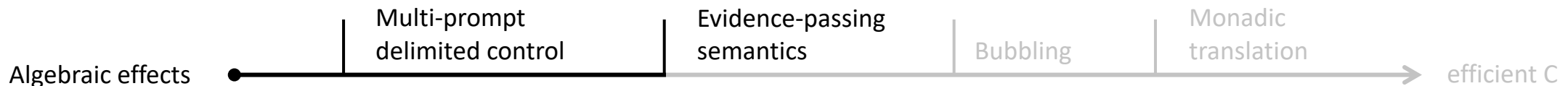
$\langle\langle\rangle\rangle$ $\langle\langle \text{read}: (m1, h1) \rangle\rangle$

prompt m1 h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (perform) $\langle\langle \text{read}: (m1, h1) \rangle\rangle.\text{read} = (m1, h1)$ constant time

$\langle\langle\rangle\rangle$ $\langle\langle \text{read}: (m1, h1) \rangle\rangle$

prompt m1 h1 (**yield** m1 (\k. f () k) + **perform** ask ())



Optimization of tail-resumptive operations

avoid yields: evaluate tail-resumptive operations in-place

```
f = \x.\k. k 1
h1 = ask -> f
```

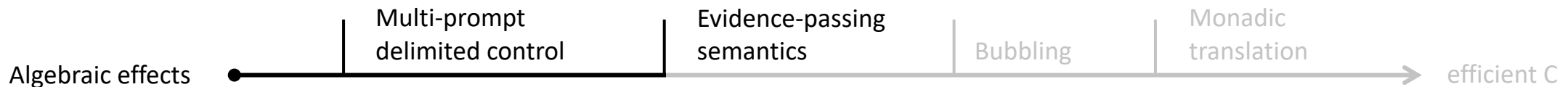
handler h1 (_. **perform** ask () + **perform** ask ())

→ (handler) (app)

<<>>
prompt m1 h1 (**perform** ask () + **perform** ask ())
<< read: (m1, h1) >>

→ (perform) << read: (m1, h1) >>.read = (m1, h1)

<<>>
prompt m1 h1 (**yield** m1 (\k. f () k) + **perform** ask ())
<< read: (m1, h1) >>



Optimization of tail-resumptive operations

avoid yields: evaluate tail-resumptive operations in-place

$f = \lambda x. \lambda k. k \text{ 1}$
 $h1 = ask \rightarrow f$

handler h1 ($\lambda _.$ **perform** ask () + **perform** ask ())

$\mapsto (handler) (app)$

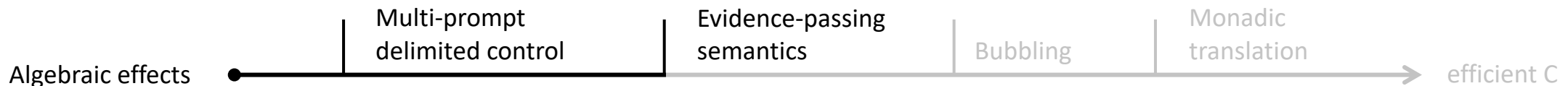
$\langle\langle \rangle\rangle$ $\langle\langle read: (m1, h1) \rangle\rangle$
prompt m1 h1 (**perform** ask () + **perform** ask ())

$\mapsto (perform) \langle\langle read: (m1, h1) \rangle\rangle.read = (m1, h1)$

$\langle\langle \rangle\rangle$ $\langle\langle read: (m1, h1) \rangle\rangle$
prompt m1 h1 (**yield** m1 ($\lambda k. f () k$) + **perform** ask ())

tail-resumptive operations

$op \mapsto \lambda x. \lambda k. k e$
 where $k \notin \text{fv}(e)$



Optimization of tail-resumptive operations

avoid yields: evaluate tail-resumptive operations in-place

$$f = \lambda x. \lambda k. k \text{ 1}$$

$$h1 = ask \rightarrow f$$

handler h1 (_ . **perform** ask () + **perform** ask ())

$\mapsto (handler) (app)$

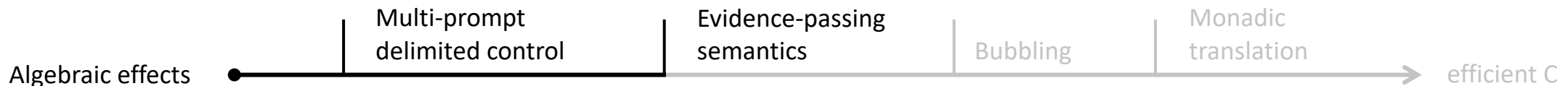
$\langle\langle \rangle\rangle$ $\langle\langle read: (m1, h1) \rangle\rangle$
prompt m1 h1 (**perform** ask () + **perform** ask ())

$\mapsto (perform) \langle\langle read: (m1, h1) \rangle\rangle.read = (m1, h1)$

$\langle\langle \rangle\rangle$ $\langle\langle read: (m1, h1) \rangle\rangle$
prompt m1 h1 (1 + **perform** ask ())

tail-resumptive operations

$op \mapsto \lambda x. \lambda k. k e$
 where $k \notin \text{fv}(e)$



Optimization of tail-resumptive operations

avoid yields: evaluate tail-resumptive operations in-place

```
f = \x.\k. k 1
h1 = ask -> f
```

handler h1 (_. **perform** ask () + **perform** ask ())

$\mapsto$ (handler) (app)

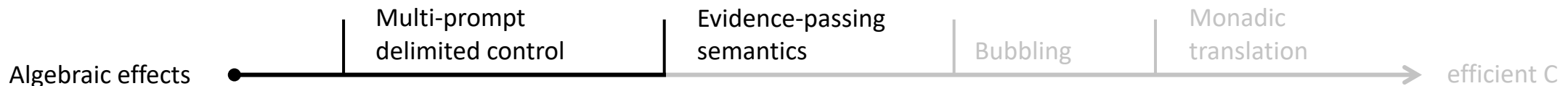
$\langle\langle \rangle\rangle$ $\langle\langle \text{read}: (m1, h1) \rangle\rangle$
prompt m1 h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (perform) $\langle\langle \text{read}: (m1, h1) \rangle\rangle$.read = (m1, h1)

$\langle\langle \rangle\rangle$ $\langle\langle \text{read}: (m1, h1) \rangle\rangle$
prompt m1 h1 (**under** read 1 + **perform** ask ())

tail-resumptive operations

$op \mapsto \lambda x. \lambda k. k e$
 where $k \notin \text{fv}(e)$



Optimization of tail-resumptive operations

avoid yields: evaluate tail-resumptive operations in-place

```
f = \x.\k. k 1
h1 = ask -> f
```

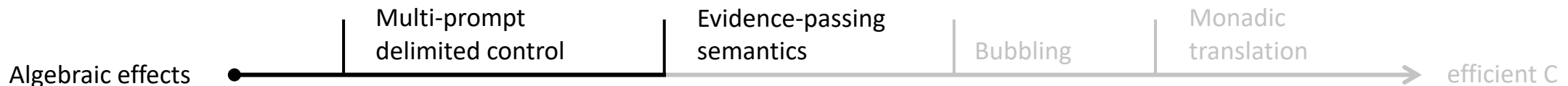
handler h1 (_. **perform** ask () + **perform** ask ())

$\mapsto$ (handler) (app)

$\langle\langle\rangle\rangle$ $\langle\langle \text{read}: (m1, h1) \rangle\rangle$
prompt m1 h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (perform) $\langle\langle \text{read}: (m1, h1) \rangle\rangle$.read = (m1, h1)

$\langle\langle\rangle\rangle$ $\langle\langle\rangle\rangle$ $\langle\langle \text{read}: (m1, h1) \rangle\rangle$
prompt m1 h1 (**under** read 1 + **perform** ask ())



Bubbling yields

make yields local: bubbling it up until it meets its corresponding prompt frame

```
handler h1 (\_. perform ask () + perform ask ())
```

```
f = \x.\k. k 1  
h1 = ask -> f
```

→ *(handler)* (*app*)

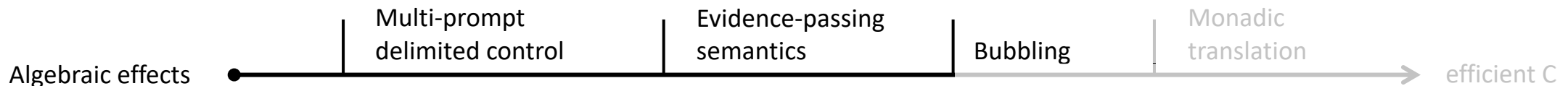
```
prompt m1 h1 (perform ask () + perform ask ())
```

→ *(perform)* searching

```
prompt m1 h1 (yield m1 (\k. f () k) + perform ask ())
```

→ *(prompt)* capturing

```
f () (\x. prompt m1 h1 (x + perform ask ()))
```



Bubbling yields

make yields local: bubbling it up until it meets its corresponding prompt frame

```
handler h1 (\_. perform ask () + perform ask ())
```

```
f = \x.\k. 1 + k 1  
h1 = ask -> f
```

→ *(handler)* (*app*)

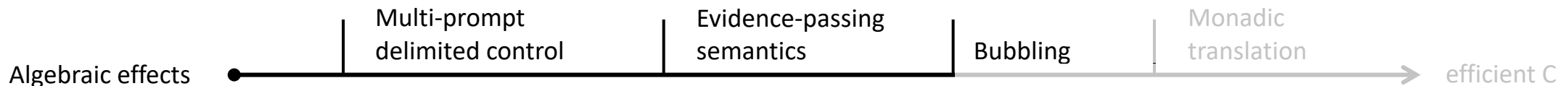
```
prompt m1 h1 (perform ask () + perform ask ())
```

→ *(perform)* searching

```
prompt m1 h1 (yield m1 (\k. f () k) + perform ask ())
```

→ *(prompt)* capturing

```
f () (\x. prompt m1 h1 (x + perform ask ()))
```



Bubbling yields

make yields local: bubbling it up until it meets its corresponding prompt frame

```
handler h1 (\_. perform ask () + perform ask ())
```

```
f = \x.\k. 1 + k 1  
h1 = ask -> f
```

→ *(handler)* (*app*)

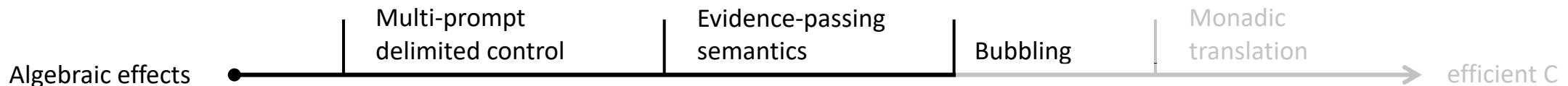
```
prompt m1 h1 (perform ask () + perform ask ())
```

→ *(perform)* searching

```
prompt m1 h1 • (+ perform ask ()) • yield m1 (\k. f () k)
```

→ *(prompt)* capturing

```
f () (\x. prompt m1 h1 (x + perform ask ()))
```



Bubbling yields

make yields local: bubbling it up until it meets its corresponding prompt frame

```
handler h1 (\_. perform ask () + perform ask ())
```

```
f = \x.\k. 1 + k 1  
h1 = ask -> f
```

→ *(handler)* (*app*)

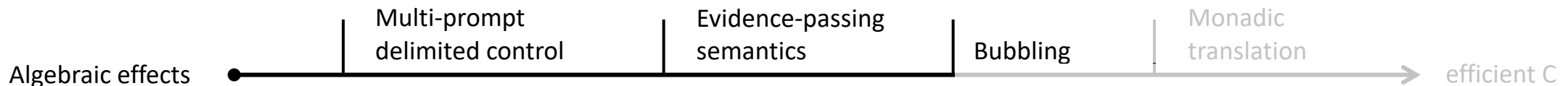
```
prompt m1 h1 (perform ask () + perform ask ())
```

→ *(perform)* searching

```
prompt m1 h1 • (+ perform ask ()) • yield m1 (\k. f () k)
```

→ *(prompt)* capturing

```
f () (\x. prompt m1 h1 • (+ perform ask ()) • x)
```



Bubbling yields

make yields local: bubbling it up until it meets its corresponding prompt frame

```
handler h1 (\_. perform ask () + perform ask ())
```

```
f = \x.\k. 1 + k 1  
h1 = ask -> f
```

→ *(handler)* (*app*)

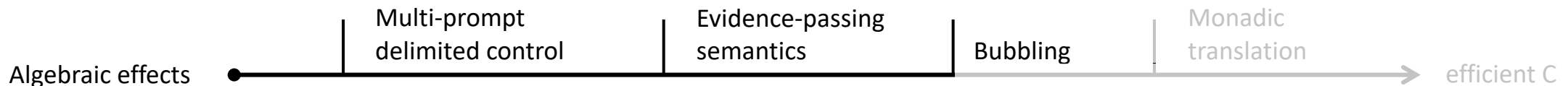
```
prompt m1 h1 (perform ask () + perform ask ())
```

→ *(perform)* searching

```
prompt m1 h1 • (+ perform ask ()) • yield m1 (\k. f () k)
```

→ *(prompt)* capturing

```
f () (\x. prompt m1 h1 • (+ perform ask ()) • x)
```



Bubbling yields

make yields local: bubbling it up until it meets its corresponding prompt frame

```
handler h1 (\_. perform ask () + perform ask ())
```

```
f = \x.\k. 1 + k 1  
h1 = ask -> f
```

→ *(handler)* (*app*)

```
prompt m1 h1 (perform ask () + perform ask ())
```

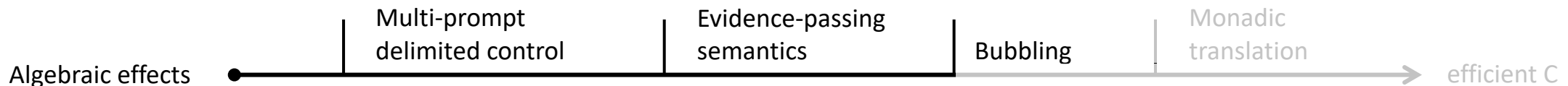
→ *(perform)* searching

```
prompt m1 h1 • (+ perform ask ()) • yield m1 (\k. f () k) (\x. x)
```

partially built-up
resumption

→ *(prompt)* capturing

```
f () (\x. prompt m1 h1 • (+ perform ask ()) • x)
```



Bubbling yields

make yields local: bubbling it up until it meets its corresponding prompt frame

handler h1 (_. **perform** ask () + **perform** ask ())

$f = \lambda x. \lambda k. 1 + k \ 1$
 $h1 = ask \rightarrow f$

→ (handler) (app)

prompt m1 h1 (**perform** ask () + **perform** ask ())

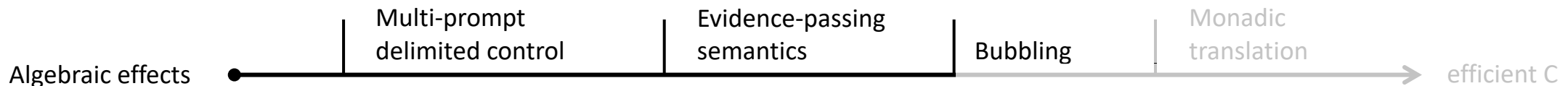
→ (perform) searching

prompt m1 h1 • (+ **perform** ask ()) • **yield** m1 (\k. f () k) (\x. x)

partially built-up
resumption

→ (prompt) capturing

f () (\x. **prompt** m1 h1 • (+ **perform** ask ()) • x)



Bubbling yields

make yields local: bubbling it up until it meets its corresponding prompt frame

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$f = \backslash x.\backslash k. \mathbf{1} + k \mathbf{1}$
 $h1 = ask \rightarrow f$

$\mapsto$ (*handler*) (*app*)

prompt m1 h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (*perform*) searching

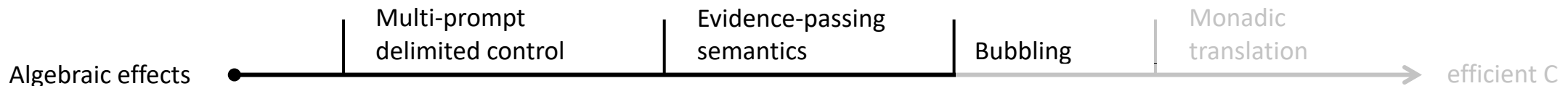
prompt m1 h1 • (+ **perform** ask ()) • **yield** m1 ($\backslash k. f$ () k) ($\backslash x. x$)

partially built-up
resumption

$\mapsto$ (*app*₁) bubbling

$\mapsto$ (*prompt*) capturing

f () ($\backslash x. \mathbf{prompt}$ m1 h1 • (+ **perform** ask ()) • x)



Bubbling yields

make yields local: bubbling it up until it meets its corresponding prompt frame

handler h1 ($\backslash_.$ **perform** ask () + **perform** ask ())

$f = \backslash x.\backslash k. \mathbf{1} + k \mathbf{1}$
 $h1 = ask \rightarrow f$

$\mapsto$ (*handler*) (*app*)

prompt m1 h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (*perform*) searching

prompt m1 h1 • (+ **perform** ask ()) • **yield** m1 ($\backslash k. f$ () k) ($\backslash x. x$)

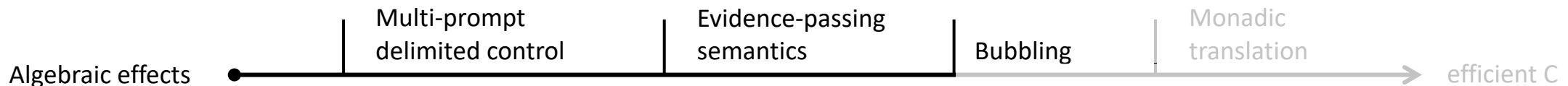
partially built-up
resumption

$\mapsto$ (*app*₁) bubbling

prompt m1 h1 •

$\mapsto$ (*prompt*) capturing

f () ($\backslash x. \mathbf{prompt}$ m1 h1 • (+ **perform** ask ()) • x)



Bubbling yields

make yields local: bubbling it up until it meets its corresponding prompt frame

handler h1 (_. **perform** ask () + **perform** ask ())

$f = \lambda x. \lambda k. 1 + k \ 1$
 $h1 = ask \rightarrow f$

$\mapsto$ (*handler*) (*app*)

prompt m1 h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (*perform*) searching

prompt m1 h1 • (+ **perform** ask ()) • **yield** m1 (\k. f () k) (\x. x)

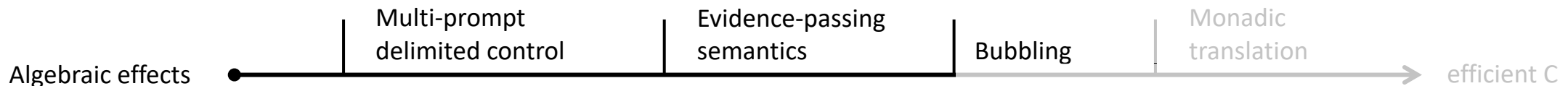
partially built-up
resumption

$\mapsto$ (*app*₁) bubbling

prompt m1 h1 • **yield** m1 (\k. f () k) (\x. (+ **perform** ask ()) • x)

$\mapsto$ (*prompt*) capturing

f () (\x. **prompt** m1 h1 • (+ **perform** ask ()) • x)



Bubbling yields

make yields local: bubbling it up until it meets its corresponding prompt frame

handler h1 (_. **perform** ask () + **perform** ask ())

$f = \lambda x. \lambda k. 1 + k \ 1$
 $h1 = ask \rightarrow f$

$\mapsto$ (*handler*) (*app*)

prompt m1 h1 (**perform** ask () + **perform** ask ())

$\mapsto$ (*perform*) searching

prompt m1 h1 • (+ **perform** ask ()) • **yield** m1 (\k. f () k) (\x. x)

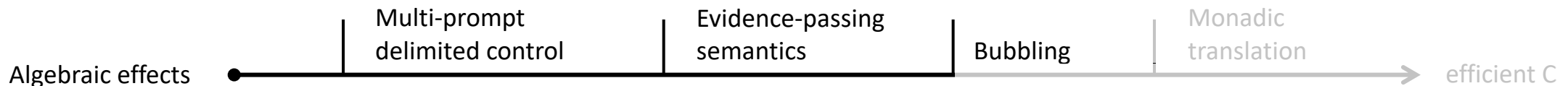
partially built-up
resumption

$\mapsto$ (*app*₁) bubbling

prompt m1 h1 • **yield** m1 (\k. f () k) (\x. (+ **perform** ask ()) • x)

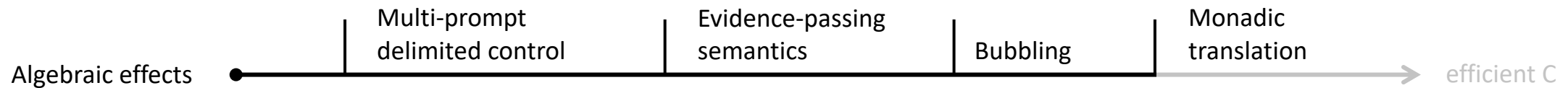
$\mapsto$ (*prompt*) capturing

f () (\x. **prompt** m1 h1 • (+ **perform** ask ()) • x)



Monadic translation

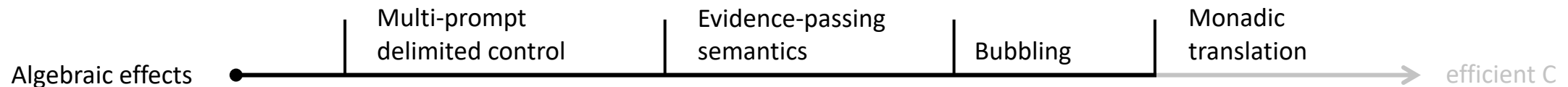
all transitions are local: translate algebraic effects into a pure lambda calculus with a multi-prompt delimited control monad



Monadic translation

all transitions are local: translate algebraic effects into a pure lambda calculus with a multi-prompt delimited control monad

```
handler h1 ( \_. perform ask ( ) + perform ask ( ) )
```

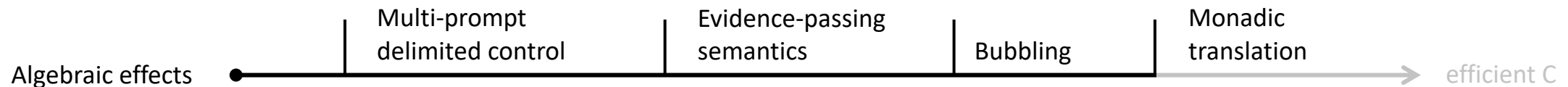


Monadic translation

all transitions are local: translate algebraic effects into a pure lambda calculus with a multi-prompt delimited control monad

```
handler h1 ( \_. perform ask ( ) + perform ask ( ) )
```

$\rightsquigarrow$



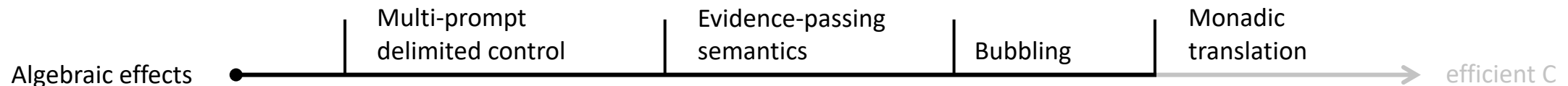
Monadic translation

all transitions are local: translate algebraic effects into a pure lambda calculus with a multi-prompt delimited control monad

```
handler h1 ( \_. perform ask () + perform ask () )
```

$\rightsquigarrow$

```
handler h1 ( \_. perform ask ()  $\triangleright$  ( \x. perform ask ()  $\triangleright$  ( \y. Pure (x + y) ) ) )
```



Monadic translation

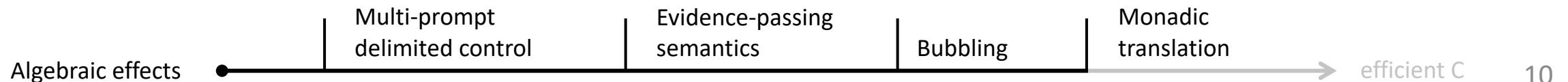
all transitions are local: translate algebraic effects into a pure lambda calculus with a multi-prompt delimited control monad

```
handler h1 ( \_. perform ask () + perform ask () )
```

$\rightsquigarrow$

```
handler h1 ( \_. perform ask ()  $\triangleright$  ( \x. perform ask ()  $\triangleright$  ( \y. Pure (x + y) ) ) )
```

A evidence-passing multi-prompt delimited control monad

$$\text{type Mon } \mu \alpha = \text{Evv } \mu \rightarrow \text{Ctl } \mu \alpha$$


Monadic translation

all transitions are local: translate algebraic effects into a pure lambda calculus with a multi-prompt delimited control monad

```
handler h1 (\_. perform ask () + perform ask ())
```

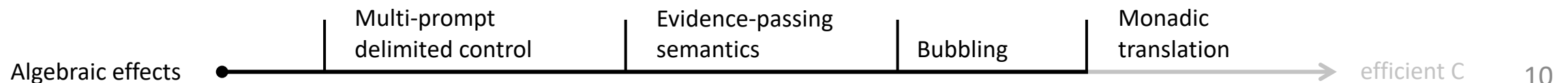
$\rightsquigarrow$

```
handler h1 (\_. perform ask () ▷ (\x. perform ask () ▷ (\y. Pure (x + y))))
```

A evidence-passing multi-prompt delimited control monad

evidence passing

type $\text{Mon } \mu \alpha = \boxed{\text{Evv } \mu \rightarrow} \text{Ctl } \mu \alpha$



Monadic translation

all transitions are local: translate algebraic effects into a pure lambda calculus with a multi-prompt delimited control monad

```
handler h1 (\_. perform ask () + perform ask ())
```

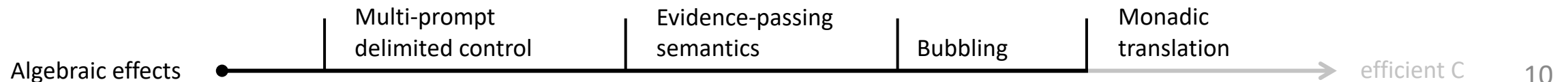
$\rightsquigarrow$

```
handler h1 (\_. perform ask () ▷ (\x. perform ask () ▷ (\y. Pure (x + y))))
```

A evidence-passing multi-prompt delimited control monad

evidence passing

type $\text{Mon } \mu \alpha = \boxed{\text{Ev} \mu \rightarrow} \boxed{\text{Ctl } \mu \alpha}$
control monad



Monadic translation

all transitions are local: translate algebraic effects into a pure lambda calculus with a multi-prompt delimited control monad

handler h1 (_. **perform** ask () + **perform** ask ())

$\rightsquigarrow$

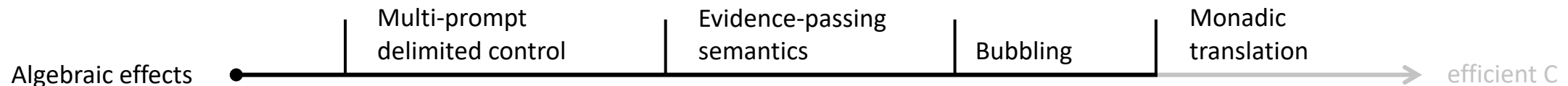
handler h1 (_. **perform** ask () $\triangleright$ (\x. **perform** ask () $\triangleright$ (\y. **Pure** (x + y))))

A evidence-passing multi-prompt delimited control monad

evidence passing

type Mon μ α = Evv $\mu \rightarrow$ Ctl μ α
control monad

$e \triangleright g$ = $\lambda w.$ case e w of Pure x $\rightarrow g$ x w
Yield m f $k \rightarrow$ Yield m f ($\lambda x.$ k $x \triangleright g$)



Monadic translation

all transitions are local: translate algebraic effects into a pure lambda calculus with a multi-prompt delimited control monad

handler h1 (_. **perform** ask () + **perform** ask ())

$\rightsquigarrow$

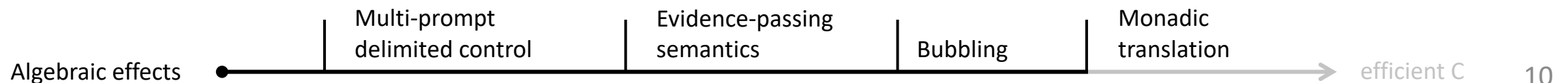
handler h1 (_. **perform** ask () $\triangleright$ (\x. **perform** ask () $\triangleright$ (\y. **Pure** (x + y))))

A evidence-passing multi-prompt delimited control monad

evidence passing

type Mon μ α = Evv $\mu \rightarrow$ Ctl μ α
control monad

$e \triangleright g$ = $\lambda w.$ case e w of Pure x $\rightarrow g$ x w ———| pass the result and the current evidence
Yield m f $k \rightarrow$ Yield m f ($\lambda x.$ k $x \triangleright g$)



Monadic translation

all transitions are local: translate algebraic effects into a pure lambda calculus with a multi-prompt delimited control monad

handler h1 (_. **perform** ask () + **perform** ask ())

$\rightsquigarrow$

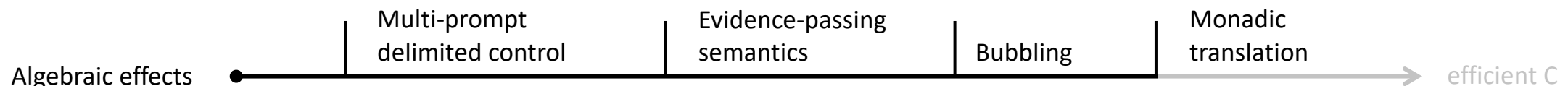
handler h1 (_. **perform** ask () $\triangleright$ (\x. **perform** ask () $\triangleright$ (\y. **Pure** (x + y))))

A evidence-passing multi-prompt delimited control monad

evidence passing

type Mon μ α = Evv $\mu \rightarrow$ Ctl μ α
control monad

$e \triangleright g$ = $\lambda w.$ case e w of Pure x $\rightarrow g$ x w ———| pass the result and the current evidence
Yield m f $k \rightarrow$ Yield m f ($\lambda x.$ k $x \triangleright g$ bubbling)

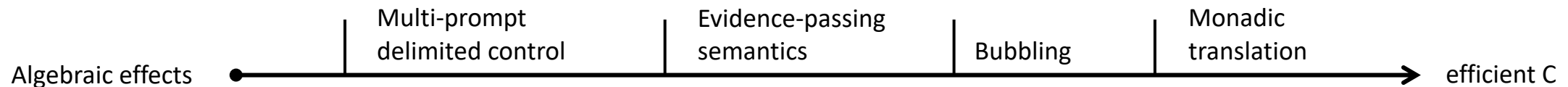


Compiling to C

```
handler h1 (\_. perform ask () + perform ask ())
```

$\rightsquigarrow$

```
handler h1 (\_. perform ask ()  $\triangleright$  (\x. perform ask ()  $\triangleright$  (\y. Pure (x + y))))
```



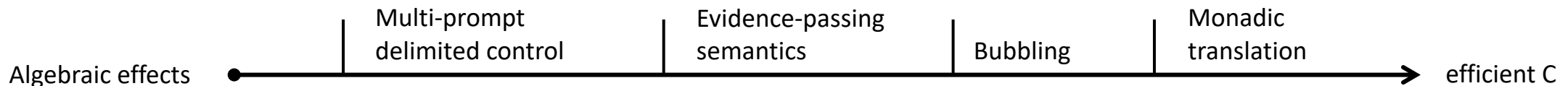
Compiling to C

```
handler h1 (\_. perform ask () + perform ask ())
```

$\rightsquigarrow$

```
handler h1 (\_. perform ask () ▷ (\x. perform ask () ▷ (\y. Pure (x + y))))
```

```
int expr( unit_t u, context_t* ctx) {  
  int x = perform_ask( ctx→w[0], unit, ctx );  
  if (ctx→is_yielding) { yield_extend(&join2,ctx); return 0; }  
  int y = perform_ask( ctx→w[0], unit, ctx );  
  if (ctx→is_yielding) { yield_extend(alloc_closure_join1(x,ctx),ctx); return 0; }  
  return (x+y); }
```



Compiling to C

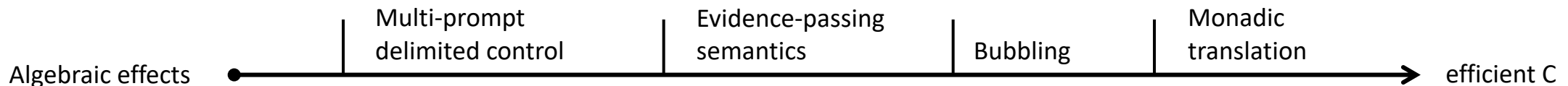
```
handler h1 (\_. perform ask () + perform ask ())
```

$\rightsquigarrow$

```
handler h1 (\_. perform ask () ▷ (\x. perform ask () ▷ (\y. Pure (x + y))))
```

evidence passing

```
int expr( unit_t u, context_t* ctx) {  
  int x = perform_ask( ctx→w[0], unit, ctx );  
  if (ctx→is_yielding) { yield_extend(&join2,ctx); return 0; }  
  int y = perform_ask( ctx→w[0], unit, ctx );  
  if (ctx→is_yielding) { yield_extend(alloc_closure_join1(x,ctx),ctx); return 0; }  
  return (x+y); }
```



Compiling to C

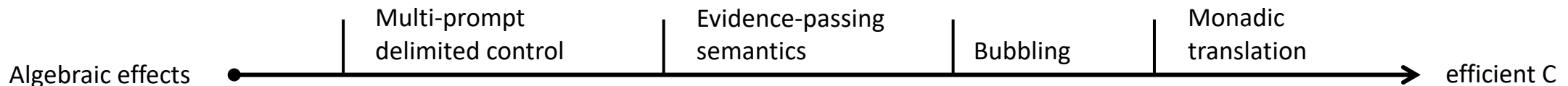
```
handler h1 (\_. perform ask () + perform ask ())
```

$\rightsquigarrow$

```
handler h1 (\_. perform ask () ▷ (\x. perform ask () ▷ (\y. Pure (x + y))))
```

evidence passing

```
int expr( unit_t u, context_t* ctx) {  
    int x = perform_ask( ctx→w[0], unit, ctx );  
    if (ctx→is_yielding) { yield_extend(&join2,ctx); return 0; }  
    int y = perform_ask( ctx→w[0], unit, ctx );  
    if (ctx→is_yielding) { yield_extend(alloc_closure_join1(x,ctx),ctx); return 0; }  
    return (x+y); }
```



Compiling to C

```
handler h1 (\_. perform ask () + perform ask ())
```

$\rightsquigarrow$

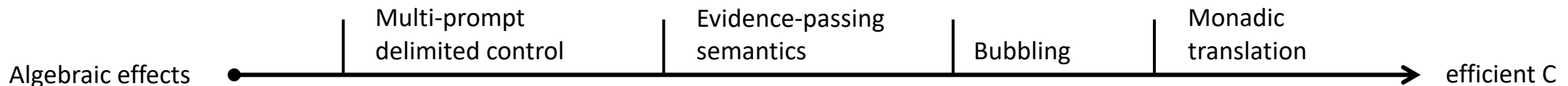
```
handler h1 (\_. perform ask () ▷ (\x. perform ask () ▷ (\y. Pure (x + y))))
```

evidence passing

```
int expr( unit_t u, context_t* ctx) {  
    int x = perform_ask( ctx→w[0], unit, ctx );  
    if (ctx→is_yielding) { yield_extend(&join2,ctx); return 0; }  
    int y = perform_ask( ctx→w[0], unit, ctx );  
    if (ctx→is_yielding) { yield_extend(alloc_closure_join1(x,ctx),ctx); return 0; }  
    return (x+y); }
```

control monad

constant-time look-up



Compiling to C

```
handler h1 (\_. perform ask () + perform ask ())
```

$\rightsquigarrow$

```
handler h1 (\_. perform ask () ▷ (\x. perform ask () ▷ (\y. Pure (x + y))))
```

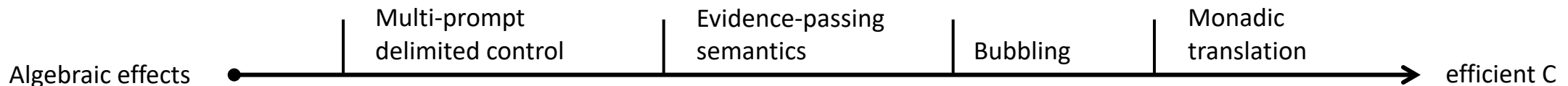
evidence passing

```
int expr( unit_t u, context_t* ctx) {  
  int x = perform_ask( ctx→w[0], unit, ctx );  
  if (ctx→is_yielding) { yield_extend(&join2,ctx); return 0; }  
  int y = perform_ask( ctx→w[0], unit, ctx );  
  if (ctx→is_yielding) { yield_extend(alloc_closure_join1(x,ctx),ctx); return 0; }  
  return (x+y); }
```

control monad

constant-time look-up

bubbling



Compiling to C

```
handler h1 (\_. perform ask () + perform ask ())
```

$\rightsquigarrow$

```
handler h1 (\_. perform ask () ▷ (\x. perform ask () ▷ (\y. Pure (x + y))))
```

evidence passing

```
int expr( unit_t u, context_t* ctx) {  
  int x = perform_ask( ctx→w[0], unit, ctx );
```

constant-time look-up

control
monad

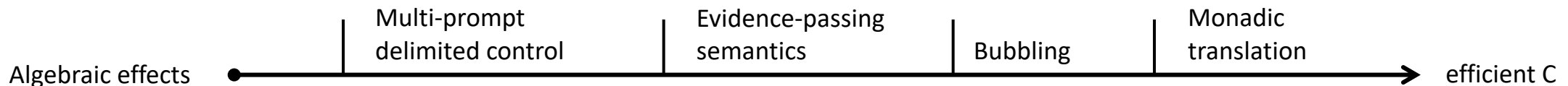
```
  if (ctx→is_yielding) { yield_extend(&join2,ctx); return 0; }
```

bubbling

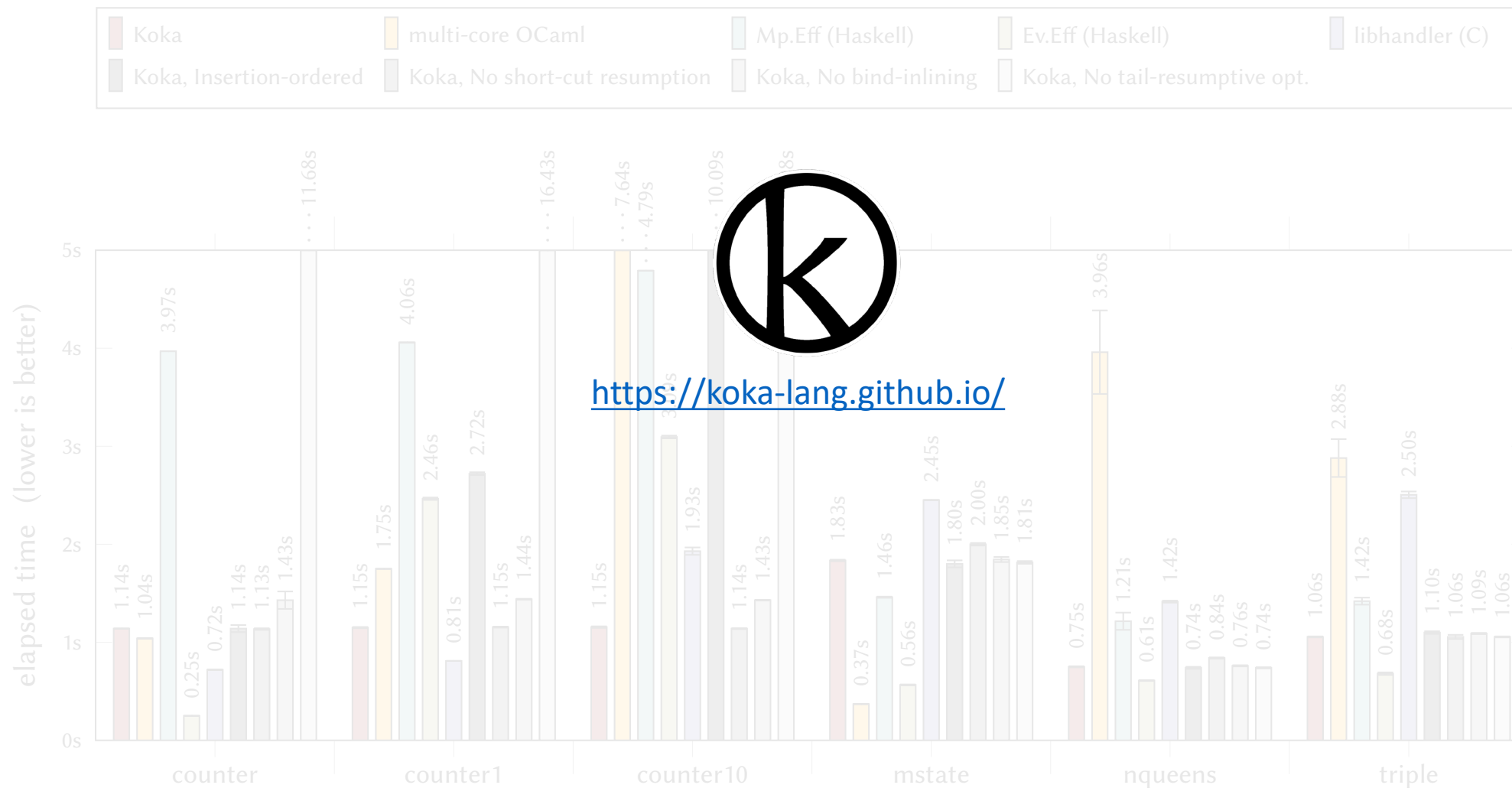
```
  int y = perform_ask( ctx→w[0], unit, ctx );
```

```
  if (ctx→is_yielding) { yield_extend(alloc_closure_join1(x,ctx),ctx); return 0; }
```

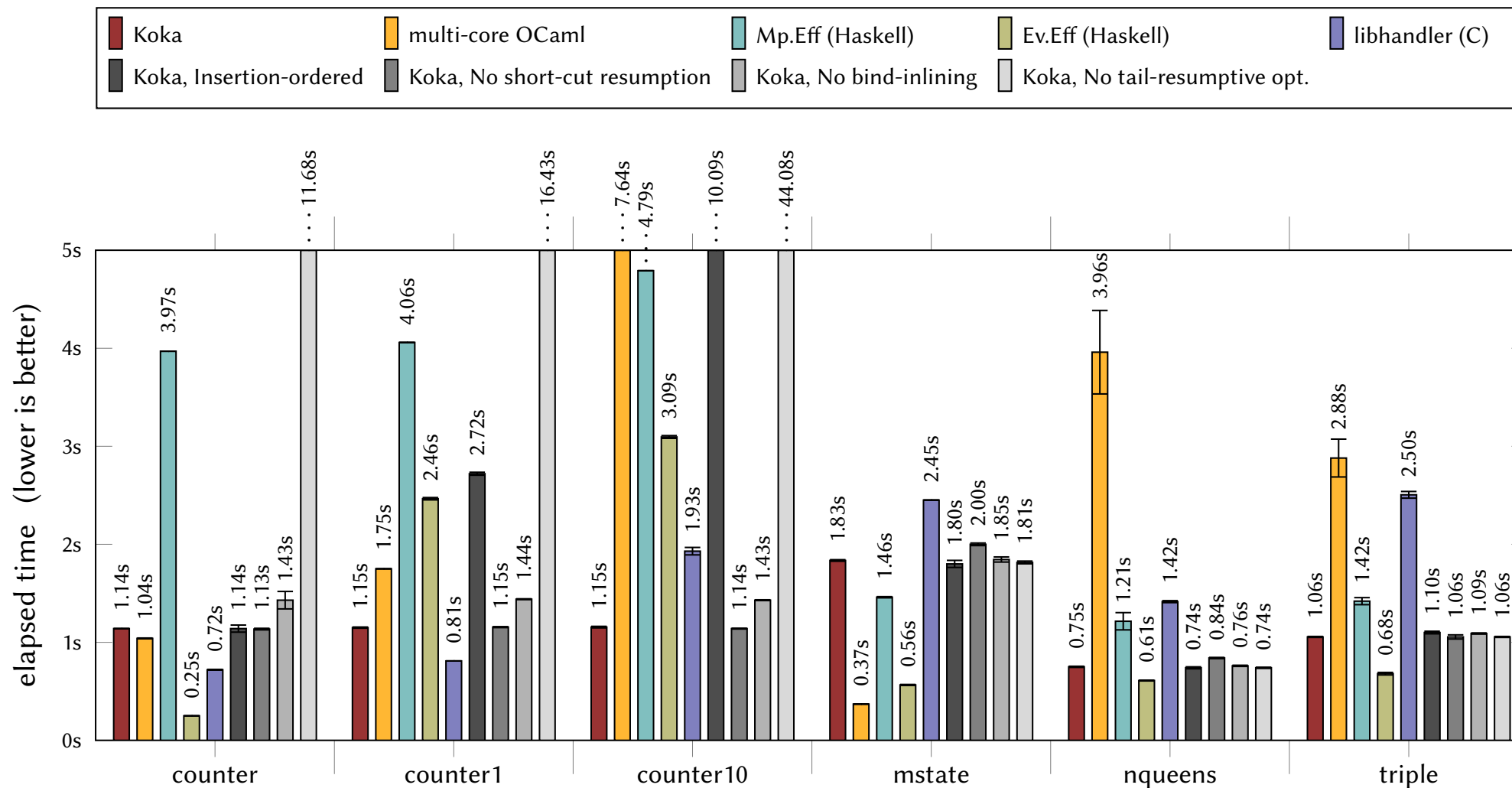
```
  return (x+y); }
```



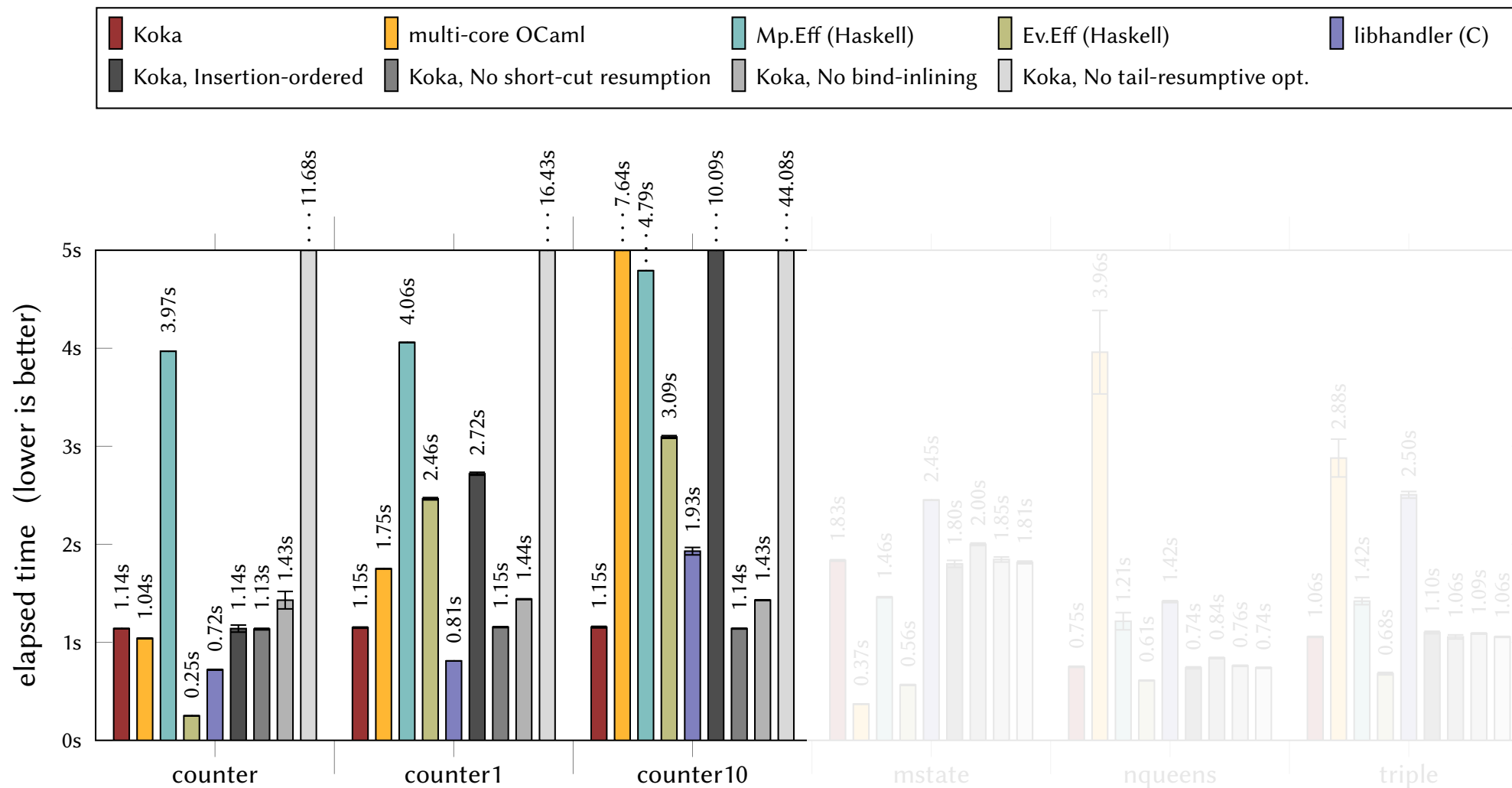
Benchmarks



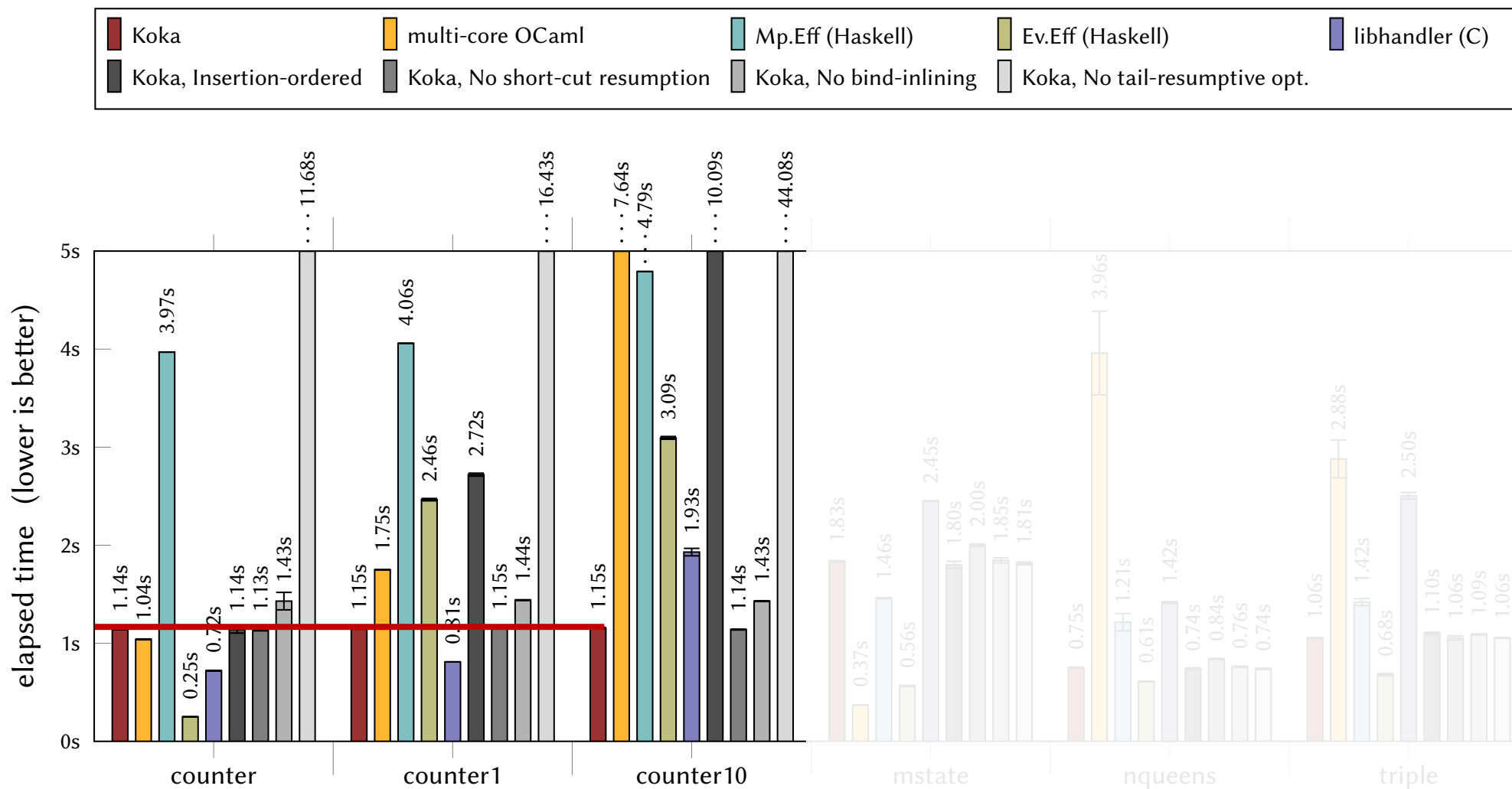
Benchmarks



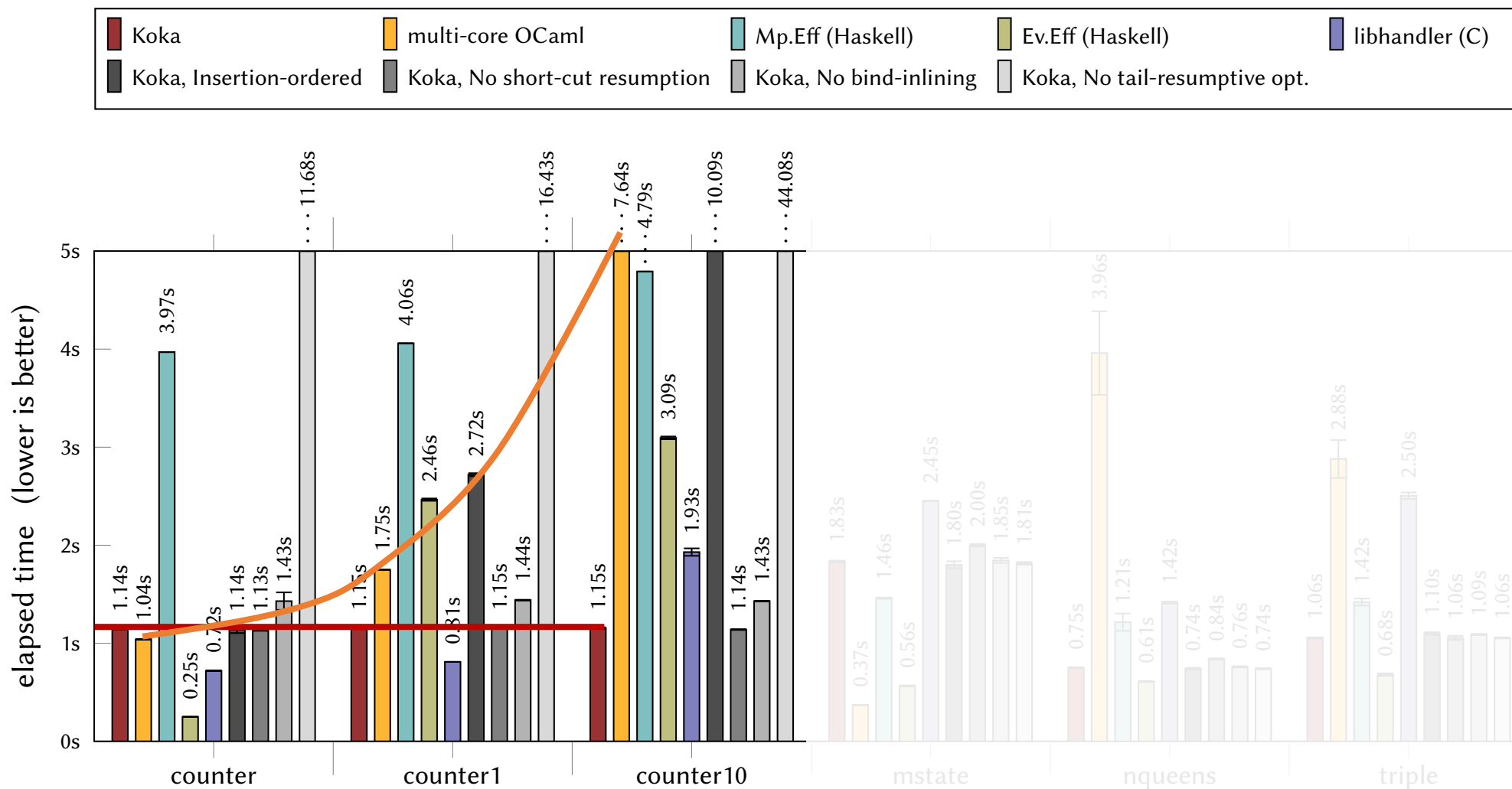
Benchmarks



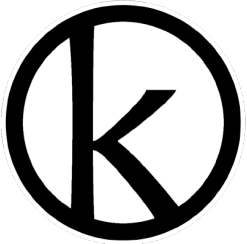
Benchmarks



Benchmarks



Excited to know more?



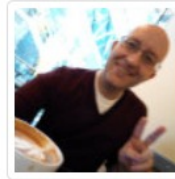
<https://koka-lang.github.io/>

Programming with Effect Handlers and FBIP in Koka

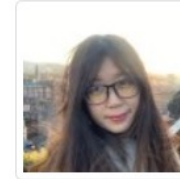
Who *Daan Leijen, Ningning Xie*

Track [ICFP 2021 Tutorials](#)

When (EST) Thu 26 Aug 2021 12:30 - 14:00 at [Tutorials](#) - [Programming with Effect Handlers and FBIP in Koka 1](#)

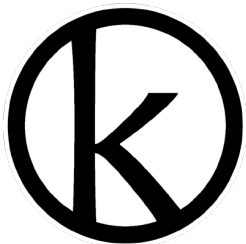


[Daan Leijen](#)
[Microsoft Research](#)
[United States](#)



[Ningning Xie](#)
[University of Hong Kong](#)
[China](#)

Excited to know more?



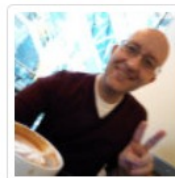
<https://koka-lang.github.io/>

Programming with Effect Handlers and FBIP in Koka

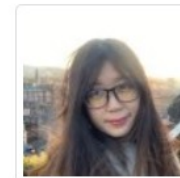
Who *Daan Leijen, Ningning Xie*

Track [ICFP 2021 Tutorials](#)

When (EST) Thu 26 Aug 2021 12:30 - 14:00 at [Tutorials](#) - [Programming with Effect Handlers and FBIP in Koka 1](#)



[Daan Leijen](#)
[Microsoft Research](#)
[United States](#)



[Ningning Xie](#)
[University of Hong Kong](#)
[China](#)



 Hackage :: [Package]

Search · [Browse](#) · [What's new](#) · [Upload](#) · [User accounts](#)

mpeff: Efficient effect handlers based on evidence-passing semantics

[[control](#), [effect](#), [library](#), [mit](#)] [[Propose Tags](#)]

See the [Control.Mp.Eff](#) module or README.md for further information

[\[Skip to Readme\]](#)

Versions [\[RSS\]](#) [\[faq\]](#)

0.1.0.0

[Change log](#)

<https://hackage.haskell.org/package/mpeff>

Generalized Evidence Passing for Effect Handlers

Efficient Compilation of Effect Handlers to C



Ningning Xie



香港大學
THE UNIVERSITY OF HONG KONG

Daan Leijen



Microsoft®
Research



ICFP 2021